

- Los esquemas de persistencia (esto es, los servicios para realizar objetos persistentes).

El ejemplo de esquema de persistencia que damos en este capítulo explicará en concreto qué es un esquema. En términos generales, un **esquema**:

- Es un conjunto cohesivo de clases que colaboran para prestar servicios a la parte fundamental e invariable de un subsistema lógico.
- Contiene clases concretas y, especialmente, abstractas que definen las interfaces a las cuales se conformarán, las interacciones en que participarán y otras invariantes.
- En términos generales, se requiere (aunque no necesariamente) que el usuario defina subclases de las actuales del esquema para que utilice, adapte y amplíe los servicios que le ofrece.
- Posee clases abstractas que pueden incluir métodos abstractos y concretos.
- Se basa en el **Principio de Hollywood**: “No nos llame, nosotros le llamamos.” Ello significa que las clases definidas por el usuario (entre ellas, las clases nuevas) recibirán mensajes de las clases previamente definidas del esquema. Suelen ser manejadas al implementar los métodos abstractos de las superclases.

El siguiente ejemplo de esquema de persistencia demostrará los principios anteriores.

38.4.1 Los esquemas son muy reutilizables

Los esquemas presentan una gran reutilización, la cual es mucho mayor que las clases individuales. Por eso, si una empresa quiere (¿y quién no?) aumentar considerablemente su grado de reúso del software, habrá de dar prioridad a la creación de esquemas.

38.5 Requerimientos del esquema de persistencia

Deseamos contar con un esquema de persistencia que ofrezca servicios a los objetos persistentes. Llamémoslo EP (esquema de persistencia). EP es un esquema simplificado; en esta introducción no estudiaremos un esquema totalmente terminado de calidad industrial, aunque sí mencionaremos algunas áreas que necesitan desarrollo u otro diseño.

En particular, el esquema debería ofrecer las siguientes funciones:

- Guardar y recuperar objetos en un mecanismo de almacenamiento persistente.
- Transacciones del tipo *commit* y *rollback*.

El diseño deberá dar soporte a las siguientes cualidades:

- Poder extenderse para soportar cualquier mecanismo de almacenamiento, como bases de datos relacionales, archivos planos, etcétera.
- Requerir un mínimo de modificación del código actual.
- Facilidad de uso (para el programador).
- Ser muy transparente: existe en el trasfondo sin forzarla.

38.6 ¿Superclase ObjetoPersistente?

Una solución parcial muy común del diseño para obtener la persistencia de los objetos consiste en crear una superclase de utilería abstracta, el *ObjetoPersistente*, que heredan todos los objetos de persistencia (figura 38.2). Es una clase que normalmente define los atributos de persistencia, entre ellos un identificador único de objetos, y también los métodos para guardar en una base de datos.

Es un procedimiento aceptable, sólo que tiene la deficiencia de acoplar firmemente la clase existente con la del *ObjetoPersistente*; las clases del dominio terminan recibiendo la herencia de una clase de utilería.

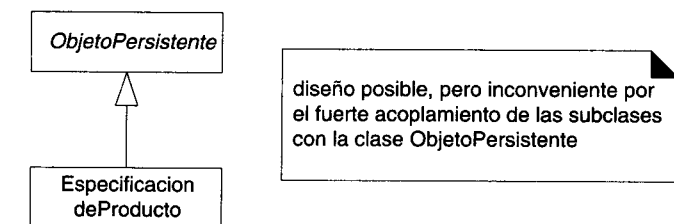


Figura 38.2 Problemas de la superclase ObjetoPersistente.

El diseño de esquema persistente examinará medios de evitar una superclase *ObjetoPersistente* y explicará las ventajas y desventajas de varios lenguajes orientados a objetos.

38.7 Ideas básicas

En secciones posteriores estudiaremos las siguientes ideas fundamentales:

- **Mapeo.** Debe haber cierto grado de mapeo entre una clase y su almacenamiento persistente (por ejemplo, la tabla de una base de datos) y entre los atributos del objeto y los campos (columnas) de un registro.
- **Identidad del objeto.** Los registros y los objetos poseen un solo identificador de objeto para relacionar fácilmente los registros con los objetos y para asegurarse de que no haya duplicados inapropiados.

- **Intermediario (broker) de base de datos.** Un agente de base de datos de fabricación pura se encarga de la materialización y de la desmaterialización.
- **Materialización y desmaterialización.** La materialización es el acto de transformar —de un almacenamiento persistente— una representación de datos no orientada a objetos (registros, por ejemplo) a objetos. Desmaterialización es la actividad opuesta (también conocida como pasivación).
- **Caché.** Los intermediarios (brokers) de bases de datos almacenan en un caché los objetos materializados.
- **Materialización lenta por demanda.** No todos los objetos se materializan inmediatamente; una instancia particular sólo se materializa por demanda cuando se necesita.
- **Referencias inteligentes.** La materialización lenta por demanda se implementa empleando una referencia inteligente denominada agente virtual.
- **Objetos complejos.** Se representan y materializan los objetos que poseen estructuras complejas, como las conexiones con muchos otros objetos.
- **Estado de la transacción del objeto.** Conviene conocer el estado de los objetos en cuanto a su relación con la transacción actual. Por ejemplo, es útil saber cuáles objetos han sido modificados (están *sucios*), con el fin de determinar si hay que volver a guardarlos y así actualizarlos en su almacenamiento persistente.
- **Operaciones de transacciones.** Son las operaciones de commit y rollback.
- **Búsqueda.** Son la localización y materialización de los objetos a partir de algunos criterios.

38.8 Mapeo: patrón *Representación de objetos como tablas*

¿Cómo mapear un objeto a un archivo o a un esquema de base de datos relacional?

El patrón **Representación de Objetos como Tablas** [Brown96] propone definir una tabla (si se emplea una base de datos relacional) para cada clase de objeto persistente. Los atributos de objetos que contienen tipos primitivos de datos (número, cadena, booleano y otros) se mapean en las columnas.

Si un objeto posee atributos exclusivamente de tipos primitivos de datos, el mapeo será simple. Pero, como veremos luego, las cosas no son tan sencillas porque los objetos pueden tener atributos que se refieran a otros objetos complejos, mientras que el modelo relacional exige que los valores sean atómicos (primera forma normal).

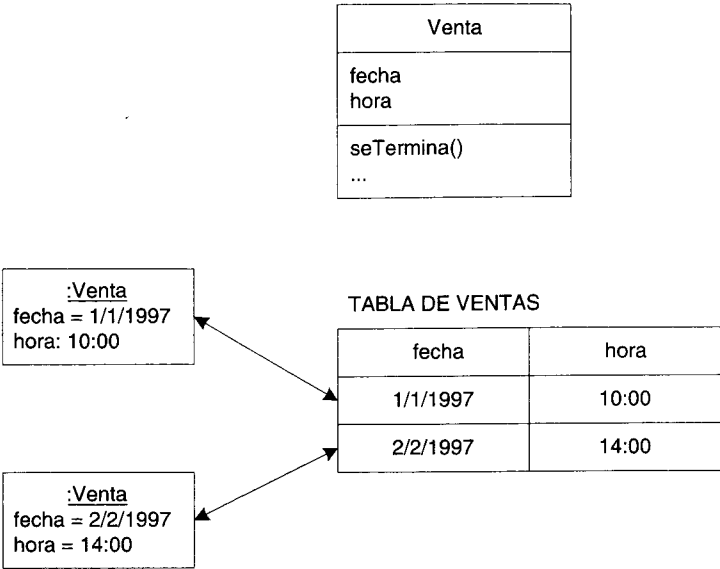


Figura 38.3 Mapeo de objetos y de tablas.

38.9 Identificación de objetos: el patrón *Identificador de objetos*

Conviene contar con un medio que relacione los objetos con los registros y de asegurarse de que la repetición de la materialización de un registro no culmine en la duplicación de objetos.¹

El patrón **Identificador de Objetos** (IDO) [Brown96] se propone asignar un IDO a cada registro y objeto (o al agente de un objeto).

Un identificador de objetos suele ser un valor alfanumérico; es único para un objeto específico. Una elección excelente es el Identificador Universalmente Único² (IUU) de 32 caracteres, que no se repite en el planeta Tierra en ninguna fecha ni hora. En reconocimiento de que nuestro planeta no representa al universo, se le cambió el nombre y se le puso otro correspondiente a la terminología de Microsoft: Identificador *Globalmente* Único (IGU), y el API Windows de Microsoft contiene una función para generarlo automáticamente.

Toda tabla de base de datos relacional tiene un IDO como clave primaria, y los objetos también contarán (directa o indirectamente) con un identificador.

Si todos los objetos se asocian a un IDO y si todas las tablas poseen una clave primaria IDO, los objetos podrán mapearse de modo singular en un renglón de alguna tabla.

¹ Más adelante examinaremos los problemas de duplicación.
² Según un algoritmo de la Open Software Foundation.

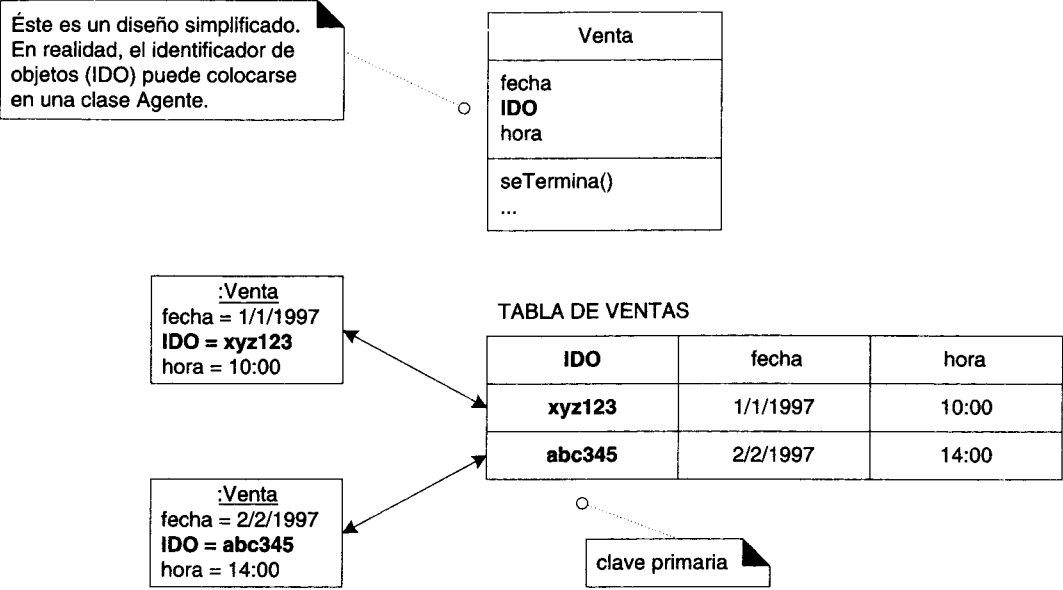


Figura 38.4 Los identificadores de objetos vinculan objetos y registros.

Esta es una visión simplificada del diseño. En realidad, el IDO no ha sido colocado en el objeto persistente, aunque es posible hacerlo. Por el contrario, podemos ponerlo en un objeto Agente, según veremos más adelante. La elección del lenguaje influye en el diseño.

38.10 Intermediarios: el patrón *Intermediario de base de datos*

¿Quién debería asumir la responsabilidad de materializar y desmaterializar los objetos (por ejemplo, una *EspecificaciondeProducto*) a partir de un almacenamiento persistente? El patrón Experto indica que debería hacerlo la clase de objeto persistente (*EspecificaciondeProducto*), pero ésta tiene varios defectos, entre otros:

- Acoplar la clase de objeto persistente con el conocimiento de almacenamiento persistente (violación del patrón Bajo Acoplamiento).
- Las responsabilidades complejas en un área nueva y sin relación con las responsabilidades que asumía antes el objeto (violación del patrón Alta Cohesión).

La solución, como siempre que el acoplamiento es grande, está constituida por el patrón Indirección: intercalar algo, generalmente una clase de Fabricación Pura.

El patrón **Intermediario de base de datos** [Brown96] propone construir una clase que se encargue de materializar, de desmaterializar y guardar un objeto en un objeto caché. Podemos definir una clase de intermediarios para cada clase de objetos persistentes.

tes.¹ Un Intermediario de base de datos es una clase de Fabricación Pura que soporta los patrones Alta Cohesión y Baja Cohesión por medio de Indirección. La figura 38.5 muestra visualmente que cada objeto persistente puede tener su propia clase de intermediario y que los mecanismos de almacenamiento pueden contar con varias clases de intermediarios.

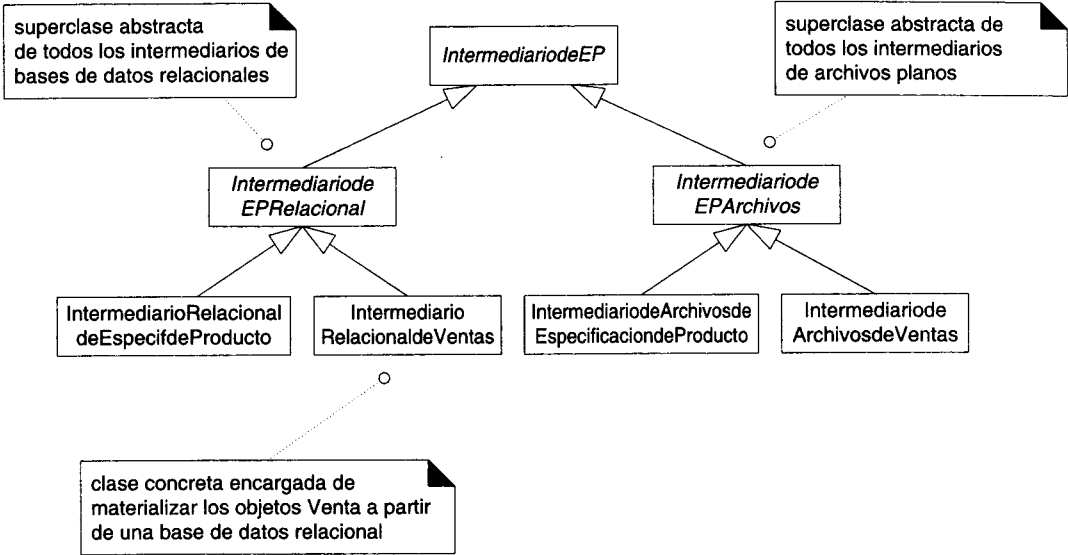


Figura 38.5 Jerarquía de intermediarios de la base de datos.

38.11 Diseño de esquemas: el patrón *Método de Plantilla*

En la siguiente sección describimos algunas de las características esenciales del diseño de Intermediarios de Bases de Datos, que forman parte importante del esquema de persistencia. Esas características se fundan en el patrón **Método de plantilla** de la Pandilla de los Cuatro [GHJV95].

Este patrón constituye la esencia del diseño de esquemas. Con él se define un método (el Método de Plantilla) en una superclase que a su vez define el esqueleto de un algoritmo, con sus partes variables e invariables. Este método llama a otros métodos, algunos de los cuales son operaciones que pueden ser omitidas en una subclase.

Por tanto, las subclases pueden desplazar los métodos variables a fin de agregar su comportamiento propio en algunos puntos de la variabilidad.

El patrón Método de Plantilla ejemplifica el principio de Hollywood: “No nos llame, nosotros lo llamamos”. En la figura 38.6 la clase *ClaseConcreta* desplaza *operacionPrimitiva*. Se la llamará automáticamente cuando se invoque el heredado *metododePlantilla*.

¹ Más adelante mencionaremos otra alternativa que se funda en metadatos y en instancias parametrizadas.

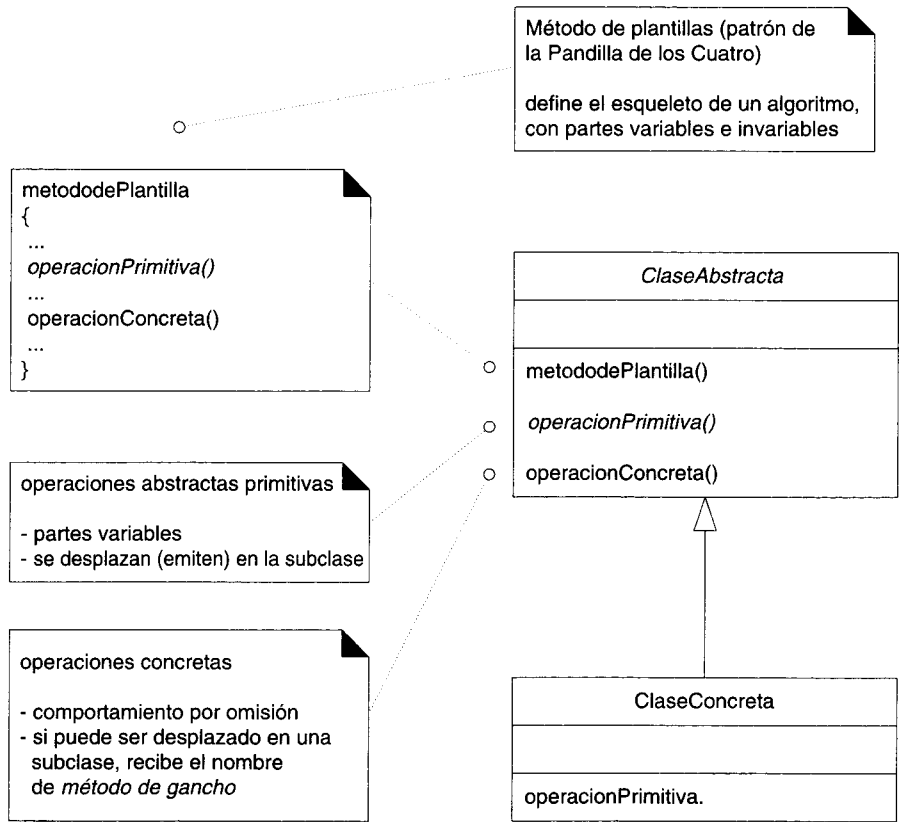


Figura 38.6 Patrón del método de plantillas.

38.12 Materialización: el patrón *Método de Plantillas*

En el esquema de persistencia, los objetos pueden recuperarse invocando el método *objetoCon(unIDO)* que se define en la clase *IntermediariodeEP(Eschema de Persistencia)*. Este método puede, a su vez, invocar a *materializarCon(unIDO)* para materializar el objeto. Como se observa en la figura 38.7, la lógica de la materialización suele requerir que se genere una instancia de la clase apropiada y que luego se desplacen los datos del registro hacia los atributos de la nueva instancia. Por ejemplo, el campo *hora* de un renglón de la tabla *VENTAS* puede copiarse en el atributo *hora* de la instancia asociada *Venta*.

La jerarquía de la clase Intermediario de Bases de Datos forma parte esencial del esquema de persistencia; el programador de la aplicación podrá agregar más clases para adoptarlas a nuevos tipos de mecanismos de almacenamiento persistente, a otras tablas o archivos dentro de un mecanismo de almacenamiento ya existente.

Por ejemplo, es posible incorporar una nueva subclase de *IntermediariodeEP* e *IntermediariodeEPArchivos* para materializar los objetos a partir de archivos planos. Y dentro

de *IntermediariodeEPRelacional* es posible agregar la materialización para otras clases, entre ellas una clase *IntermediarioRelacionaldeVentas*.

Como se muestra en la figura 38.7, el diseño expansible se funda en el patrón Método de Plantillas.¹

El método público más importante en *IntermediariodeEP* es *objetoCon(unIDO)*, método de plantilla que adopta un identificador como parámetro y luego devuelve el objeto de él. Si ya antes materializamos el objeto y actualmente está en un caché, tan sólo se devuelve. Pero si no estaba en caché, lo materializamos mediante el método *materializarCon(unIDO)*, el cual es una “operación primitiva” del método de plantilla *objetoCon*. Podemos incorporar más subclases que se materialicen en diversas formas.

Nótese la naturaleza característica del método de plantillas *objetoCon*. Define partes variables e invariables del esquema. El patrón de consulta del caché y la materialización son invariables, pero los detalles de cómo se realiza la materialización pueden modificarse; se conserva como una operación primitiva que deben definir las subclases.

De modo análogo, *registroActualComoObjeto* es una operación elemental del método de plantilla *IntermediariodeEPRelacional::materializarCon*. Un intermediario concreto para cada pareja clase/tabla define cómo transformar en un objeto el actual registro de la base de datos que ha sido leído. En este ejemplo, la transformación aparece bastante sencilla; más tarde examinaremos una lógica más compleja de la materialización.

La jerarquía de *IntermediariodeEP* muestra las tres siguientes características clásicas del diseño de esquemas:

1. El uso de los métodos de plantillas en superclases abstractas que se definieron de antemano.
2. La incorporación de las subclases definidas por el programador.
3. La definición de los métodos de “operación primitiva” en las subclases para completar los métodos de plantilla heredados.

¹ Más adelante expondremos una solución alterna que se basa en metadatos.

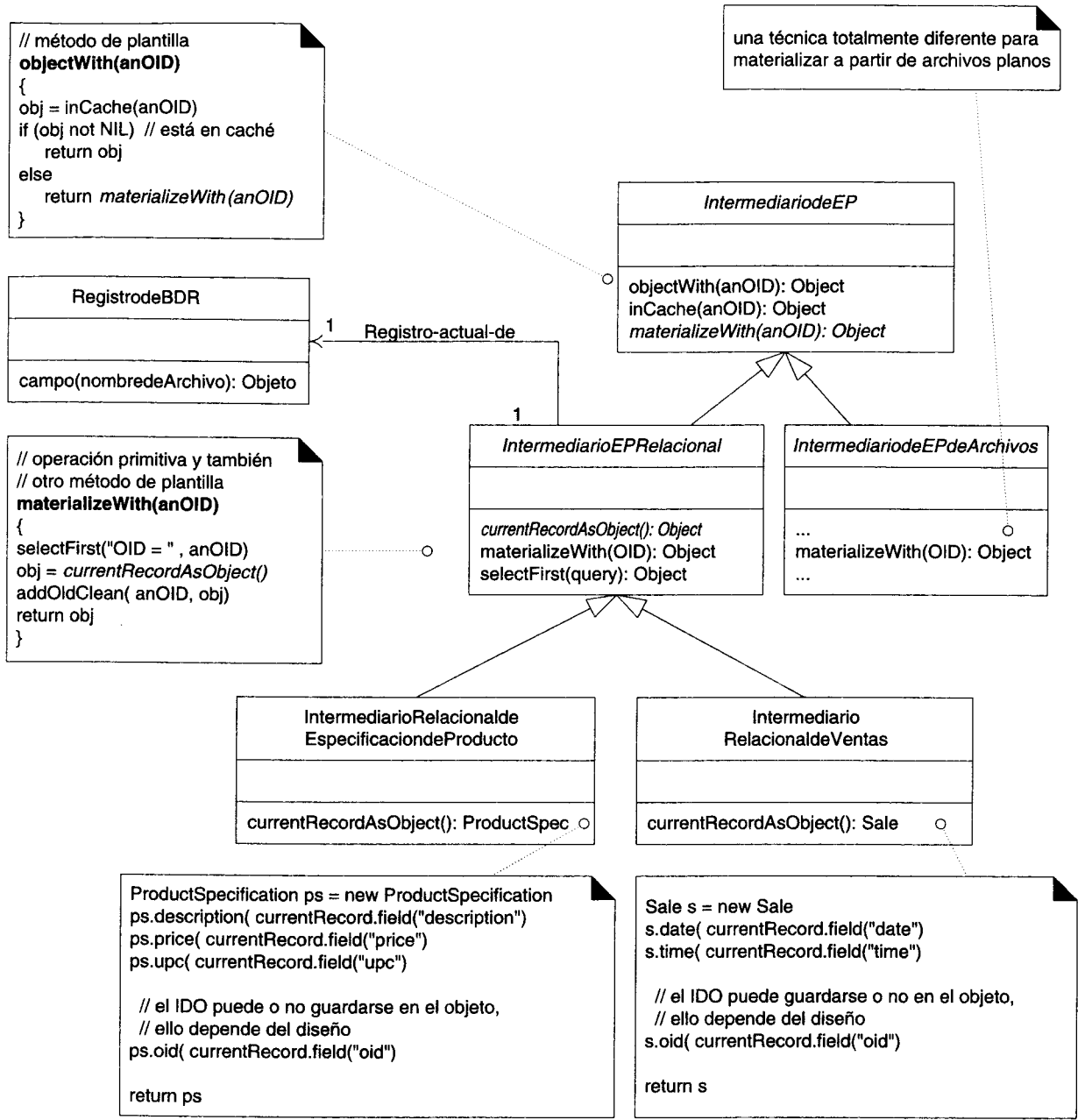


Figura 38.7 Patrón del método de plantilla en la jerarquía del Intermediario de Bases de Datos.

38.13 Objetos colocados en espacio caché: el patrón Administración de Caché

Conviene colocar en una caché local los objetos materializados, pues sólo así se podrá mejorar el desempeño (la materialización es relativamente lenta) y brindar soporte a las operaciones de la administración de transacciones, entre ellas la de commit (figura 38.8).

El patrón **Administración de Caché** [Brown96] propone asignar a los intermediarios de bases de datos la responsabilidad de dar mantenimiento a su caché. Si se utiliza un intermediario diferente con cada clase de objeto persistente, el intermediario habrá de darle mantenimiento a su propia caché.

Cuando se materializan los objetos, se colocan en la caché con su identificador como clave. Si después se le pide al intermediario un objeto, primero buscará la caché y con ello evitará una materialización innecesaria.

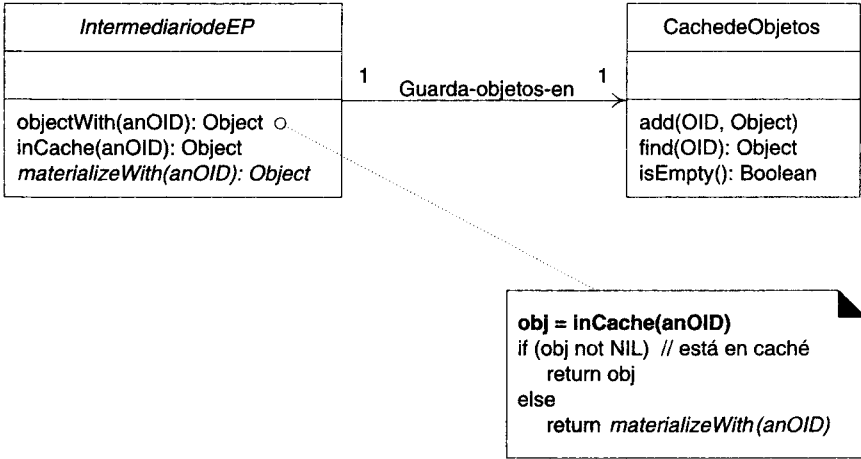


Figura 38.8 Los intermediarios conservan una caché de los objetos materializados.

38.13.1 Cachés para la administración de transacciones

Una variante de una caché individual consiste en conservar los objetos en varias cachés, según el estado que presenten dentro del contexto de la transacción actual (figura 38.9). En este método, el intermediario conserva hasta seis cachés.¹ Se sientan así las bases para determinar cómo realizar un commit y un rollback de las transacciones.

¹ En la generalidad de las aplicaciones se requieren menos de seis porque las reglas de transacción son las mismas en varias cachés. Por ejemplo, las cachés Nueva y Limpia y Nueva y Sucia se tratan igual tanto en un commit (insertar en base de datos) como en un rollback. Más adelante explicaremos una alternativa de las cachés múltiples que se basa en objetos estado.

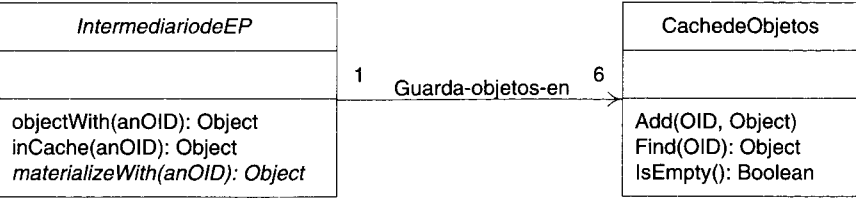


Figura 38.9 Cachés múltiples del intermediario.

Las seis cachés son:

- 1. **Caché Limpia y Nueva.** Objetos nuevos, sin modificaciones.
- 2. **Caché Limpia y Vieja.** Objetos viejos que se materializan a partir de una base de datos, sin modificaciones.
- 3. **Caché sucia y nueva.** Objetos nuevos, modificados.
- 4. **Caché sucia y vieja.** Objetos viejos que se materializaron de una base de datos y que fueron modificados.
- 5. **Caché Eliminar Nueva.** Objetos nuevos a eliminar.
- 6. **Caché Eliminar Vieja.** Objetos viejos que se materializaron a partir de una base de datos y que deben ser eliminados.

En secciones posteriores veremos cómo los objetos se dirigen hacia estas cachés y el razonamiento que se les aplica con las operaciones de commit y de rollback.

38.14 Referencias inteligentes: los patrones
Agente Virtual y Puente

Como veremos luego con mayor detalle, en ocasiones conviene posponer la materialización de un objeto hasta que sea absolutamente necesaria. A esto se le llama **materialización lenta** o **por demanda**. Podemos implementarla por medio del patrón Agente Virtual de la Pandilla de los Cuatro, una de las muchas variantes del patrón Agente [GHJV95]. (En un capítulo anterior examinamos ya Agente Remoto y Agente Dispositivo.)

Un **Agente Virtual** es un agente que hace referencia inteligente a otro objeto (el *sujeto real*) que materializa este último al referenciarlo por primera vez; implementa, pues, una materialización por demanda. Es un objeto ligero que representa a un objeto “real”, el cual puede materializarse o no. Al Agente Virtual se le considera una referencia inteligente, porque un cliente la trata como una referencia regular al sujeto real.

Un Agente Virtual es un caso especial del patrón Indirección. Como se advierte en la figura 38.10, un objeto cliente tiene una referencia al objeto Agente Virtual y no al sujeto real, pero el cliente trata al agente como si fuera el sujeto real. El Agente Virtual implementa la misma interfaz que ese sujeto; así que el objeto cliente ve al Agente Virtual como si fuera el sujeto real.

Un Agente Virtual es también una especialización de otro patrón de la Pandilla de los Cuatro: Puente, conocido también con el nombre de Manivela-Cuerpo [GHJV95]. El patrón **Puente** es también un caso especial del patrón Indirección, donde un objeto intermedio (la *Manivela*) recibe solicitudes de un cliente y las transmite a otro objeto (el *Cuerpo*) para que las atienda. El objeto Manivela suele ser muy sencillo en lo tocante a sus responsabilidades; en términos generales, se limita a transmitirle al Cuerpo las solicitudes. El patrón Puente permite al diseñador desacoplar la implementación en la interfaz.

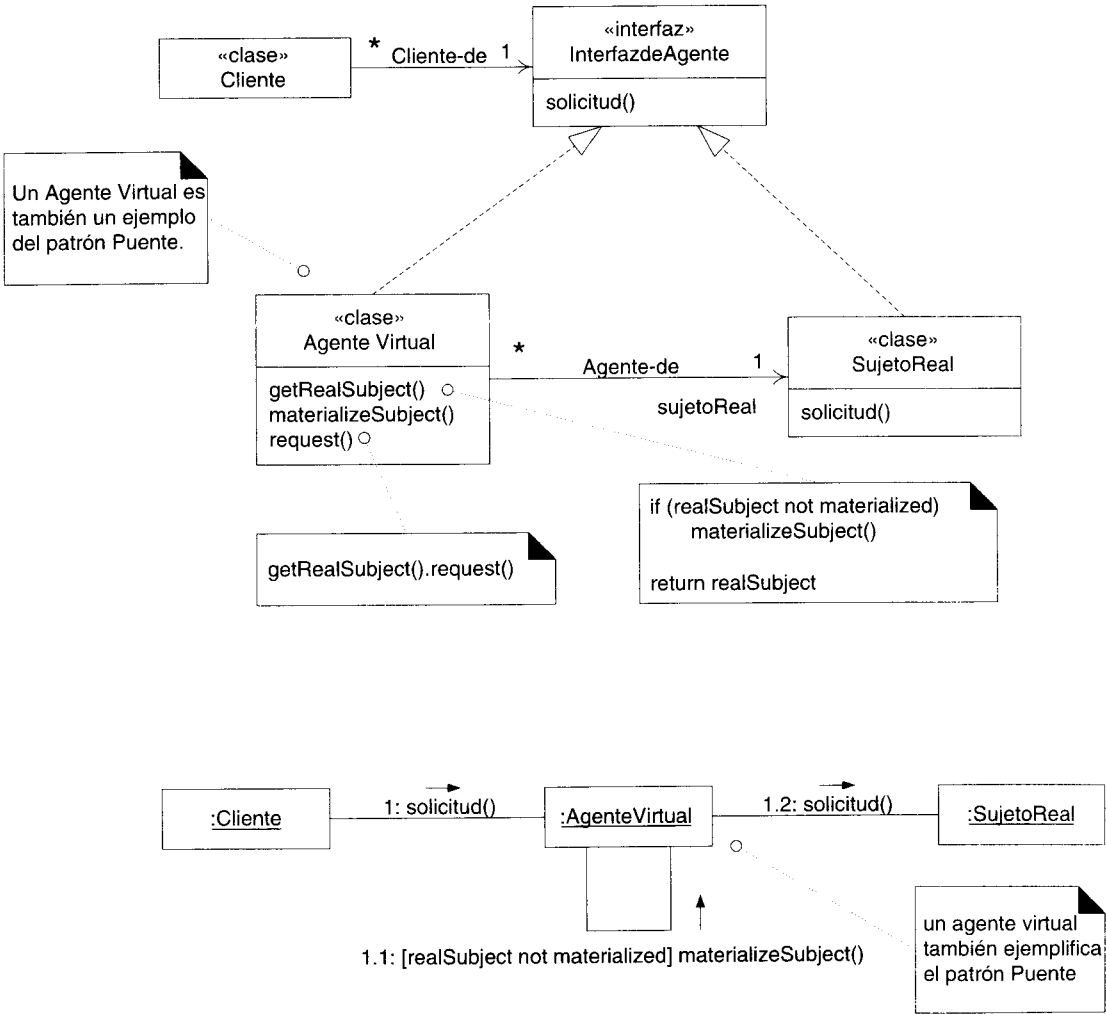


Figura 38.10 Los patrones Agente Virtual y Puente.

En la figura 38.11 se da un ejemplo concreto del patrón Agente Virtual con *VentasLineadeProducto* y con *EspecificaciondeProducto*. El diseño se basa en la suposición de que los agentes conocen el identificador de objetos de su sujeto real y de que, cuando se requiere la materialización, el identificador sirve para localizar y recuperar el sujeto real.

Nótese que *VentasLineadeProducto* posee visibilidad de atributo ante una instancia *EspecificaciondeProducto*. Esta instancia de *VentasLineadeProducto* tal vez todavía no se materialice en la memoria. Cuando *VentasLineadeProducto* envía un mensaje *descripción* al agente, materializa la *EspecificaciondeProducto* real, utilizando para ello el identificador de objeto de la *EspecificaciondeProducto* real para identificar al sujeto real.

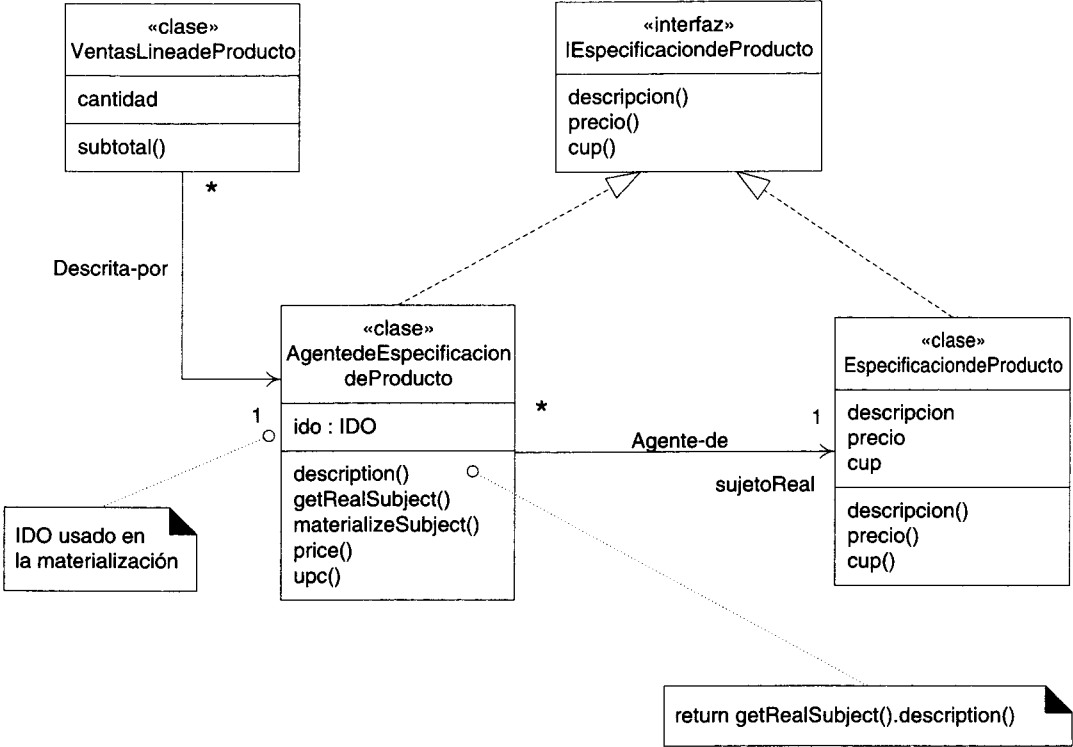


Figura 38.11 Agente Virtual de EspecificaciondeProducto.

38.14.1 Agente Virtual generalizado

Todas las clases de Agente Virtual han de efectuar la materialización por demanda, necesitan conocer su sujeto real y también pueden conocer el identificador de objetos; de ahí la posibilidad de crear una superclase abstracta generalizada *AgenteVirtual* que cumpla estas responsabilidades. Las subclases concretas tan sólo deben dar soporte a la interfaz del sujeto real que han de implementar. Este diseño se muestra visualmente en la figura 38.12.

38.14.2 No se requiere una superclase común ObjetoPersistente

Nótese que, en este diseño, no es necesario conservar el identificador de objetos en el objeto del dominio (por ejemplo, *EspecificaciondeProducto*). Gracias a ello podemos prescindir de los objetos del dominio para recibir la herencia de una superclase común como *ObjetoPersistente* (figura 38.13).

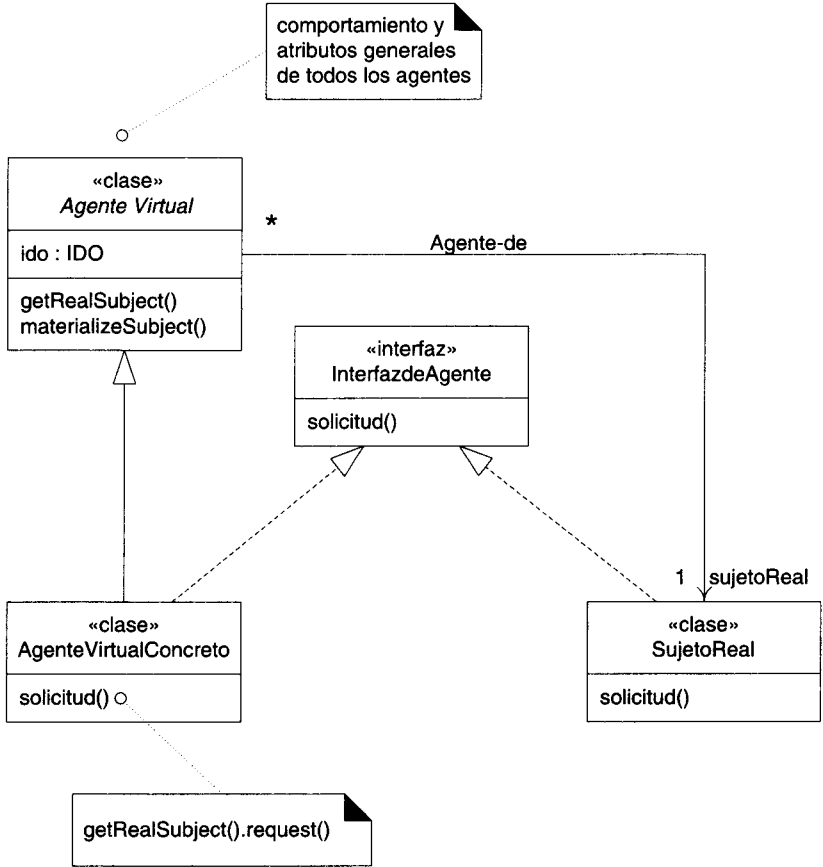


Figura 38.12 La clase generalizada Agente Virtual.

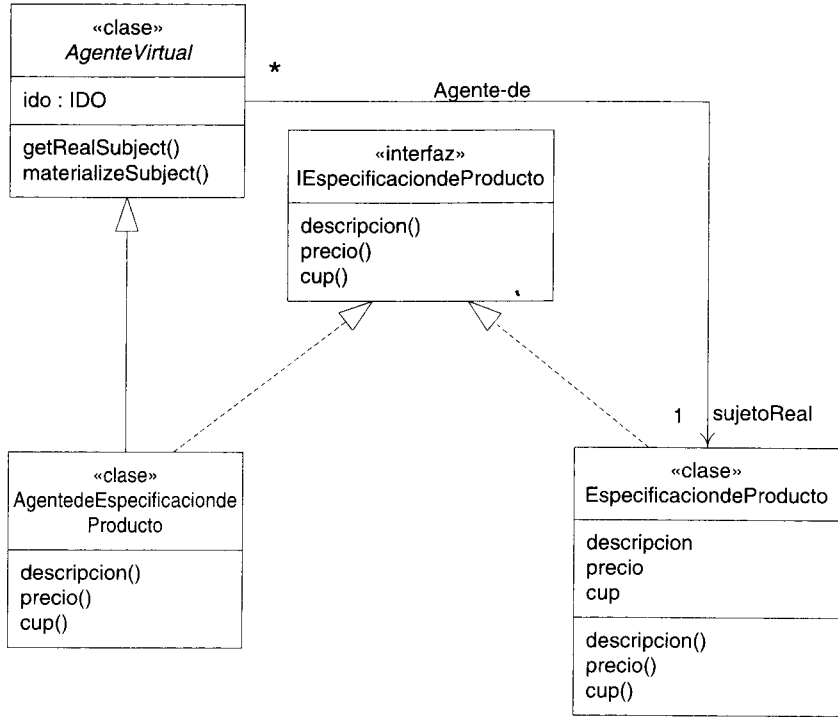


Figura 38.13 La clase *Agente de Especificacion de Producto*.

38.14.3 Implementación de un Agente Virtual

La implementación de un Agente Virtual varía según el lenguaje que se utilice. Los detalles de la implementación rebasan el ámbito de este capítulo; por ello nos limitamos a ofrecer una sinopsis:

Lenguaje	Implementación de Agente Virtual
C++	Definir una clase de apuntadores inteligentes con plantilla. No se define la clase abstracta <i>AgenteVirtual</i> ni las clases <i>AgenteVirtualConcreto</i> , pero el apuntador inteligente realiza su comportamiento. No se requiere definir <i>Interfaz de Agente</i> .

Lenguaje	Implementación de Agente Virtual
Java	Se implementa la clase <i>AgenteVirtual</i> . Se implementan las clases <i>AgenteVirtualConcreto</i> . Se definen las interfaces <i>Interfaz de Agente</i> .
Smalltalk	- Se implementa la clase <i>AgenteVirtual</i>. Se implementan las clases <i>AgenteVirtualConcreto</i>. No se requieren interfaces de <i>Agente</i> debido al vínculo dinámico de Smalltalk. - Definir un Agente virtual cambiante (o Agente Fantasma), que utiliza <i>#doesNotUnderstand:</i> y <i>#become:</i> para cambiar al sujeto real. No se definen la clase abstracta <i>AgenteVirtual</i> ni las clases <i>AgenteVirtualConcreto</i>. No se requiere la definición de <i>Interfaz de Agente</i>.

38.15 Agentes Virtuales e Intermediarios de Bases de Datos

Ahora que ya sentamos las bases de nuestra exposición, vamos a describir los detalles del esquema de persistencia. Un Agente Virtual puede colaborar con un Intermediario de Bases de Datos a fin de materializar un objeto, partiendo para ello del identificador de objetos conservado por el agente (figura 38.14).

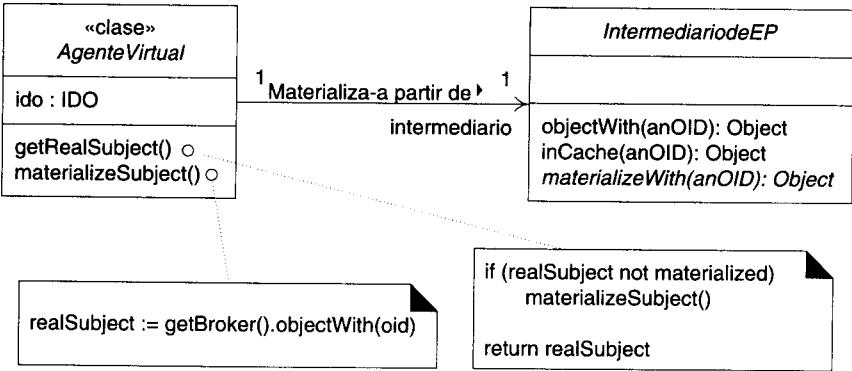


Figura 38.14 Colaboración entre Agente Virtual e intermediario de Base de Datos.

38.15.1 Conexión de Agente Virtual e Intermediario de base de datos: el patrón Método Fábrica

¿Cómo un Agente Virtual concreto sabe cuál Intermediario de Base de Datos habrá de utilizar? La respuesta consiste en aplicar el patrón Método Fábrica de la Pan-

dilla de los Cuatro [GHJV95]. El patrón **Método Fábrica** (figura 38.15) es un caso especial del patrón Método de Plantilla en que la operación primitiva (el Método Fábrica) se encarga de crear una instancia. Este método, además de su patrón más general Método de Plantilla, es muy común en el diseño de esquemas. El Método Fábrica *crearIntermediario* se sirve del patrón Singleton para devolver un Intermediario de Base de Datos, porque conviene tener una sola instancia de cada intermediario.

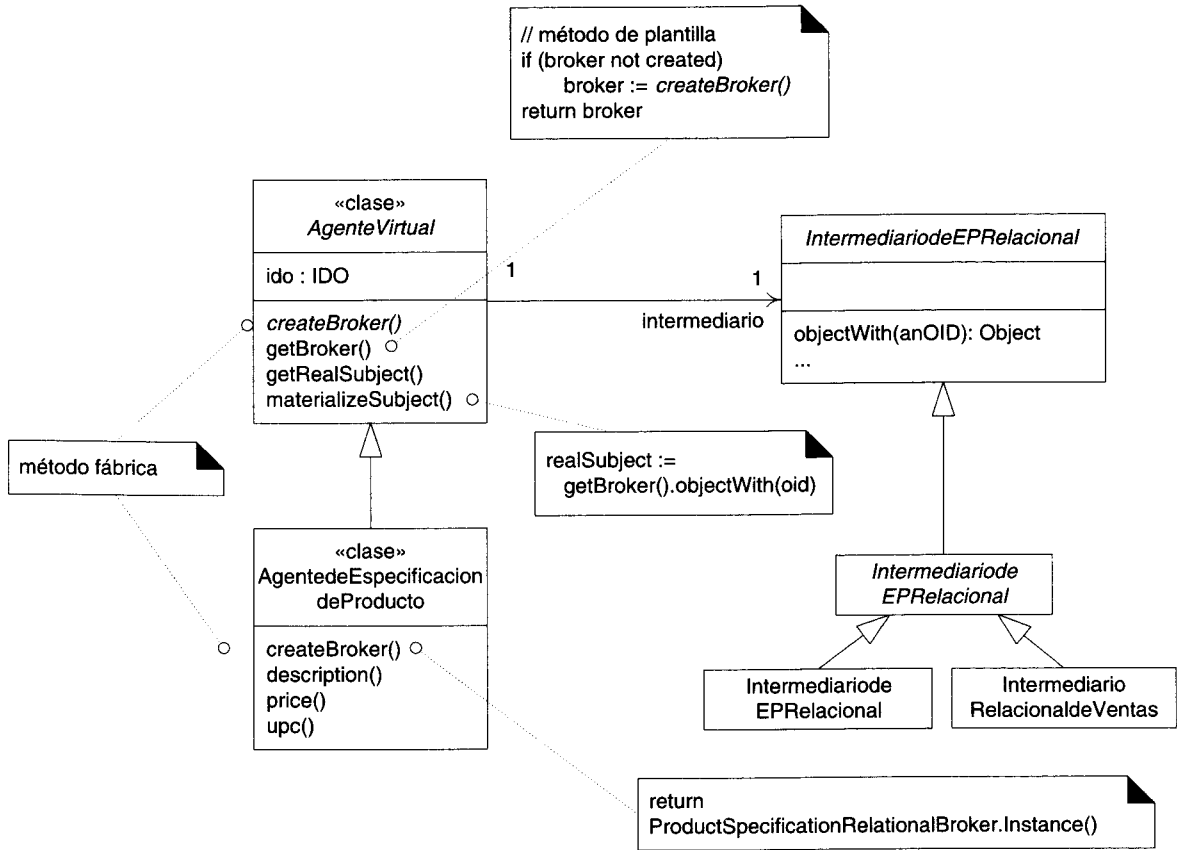


Figura 38.15 El patrón Método Fábrica.

Ahora podemos ver un tema que se repite varias veces en el diseño de esquemas: las clases *Intermediario de EP* y *AgenteVirtual* forman parte del esquema y comparten algunas cualidades con ellos. Por ejemplo:

- Las superclases abstractas pueden ser subclases para el usuario de los esquemas.
- Se definen los métodos de plantilla cuyas operaciones primitivas define el usuario de esquemas en una subclase.

38.15.2 Cómo convertir todo en un agente

Cuando se usan agentes virtuales, conviene que *toda* referencia a los objetos se efectúe a través de los objetos agente y no a través de las referencias directas. Ello significa que:

- Todas las definiciones de atributos se refieren a objetos agente a interfaces, no a objetos directos.
- Todos los parámetros se refieren a objetos agente o a interfaces.

Por ejemplo, si va a usarse el lenguaje C++ y si los agentes se implementan con apuntadores inteligentes provistos de plantillas, todos los miembros de datos y las declaraciones de parámetros estarán destinados a los apuntadores inteligentes, no a los objetos persistentes. En el caso de C++, esta técnica también puede servir para soportar la administración automática de la memoria; los agentes están en condiciones de administrarla.

Si va a usarse el lenguaje Java, todas las declaraciones de los campos de instancias y de parámetros se referirán a las interfaces (cosa que aconsejamos en todos los casos por su flexibilidad).

Esta utilización constante de los agentes garantiza que la materialización por demanda y las operaciones de transacciones se lleven a cabo correctamente.

38.16 Cómo Representar las relaciones en tablas

En la siguiente sección explicaremos cómo materializar los objetos complejos: los que se conectan a otros en vez de limitarse a conservar atributos primitivos simples. A manera de preparación para esa exposición, es necesario examinar las formas en que podemos definir las relaciones de objetos en un almacenamiento persistente. Emplearemos tablas relacionales en nuestra exposición, aunque se dispone de esquemas para otros mecanismos de almacenamiento.

¿Cómo representar las relaciones de objetos en una tabla de una base de datos relacional? La respuesta se da en el patrón **Representación de las relaciones de objetos como Tablas** [Brown96] que propone lo siguiente:

- Asociaciones **uno a uno**.
 - Colocar una clave foránea del identificador de objetos en una o en las dos tablas que representan los objetos en la relación.
 - Crear una tabla asociativa que registre los identificadores de objetos de cada objeto en la relación.
- Asociaciones de **uno a muchos**, una colección por ejemplo.
 - Crear una tabla asociativa que registre los identificadores de cada objeto en la relación.

- Asociaciones de **muchos a muchos**.
 - Crear una tabla asociativa que registre todos los identificadores de objetos en la relación.

Utilizaremos siempre una tabla asociativa para simplificar nuestra exposición.

38.17 El patrón *Instanciación de Objetos Complejos*

38.17.1 Problema: materializar una jerarquía de composición

Los objetos Agente Virtual e Intermediario de Base de Datos nos dan los medios para lograr la materialización por demanda. Entonces nos preguntamos: ¿por qué molestarnos? Lo hacemos porque los objetos pueden contener conexiones con otros, formando así una jerarquía de composición. Supongamos, por ejemplo, que los objetos *Venta*, *VentasLineadeProducto* y *EspecificaciondeProducto* son persistentes. ¿Qué significa materializar una *Venta*? ¿Significa acaso que se materializa únicamente la *Venta* o también su *VentasLineadeProducto* y su *EspecificaciondeProducto* asociada? Si queremos materializar un objeto con una profunda jerarquía de composición, posiblemente también haya que materializar docenas o cientos de objetos relacionados. La materialización de una jerarquía íntegra de composición usa el espacio lenta e ineficientemente.

38.17.2 Solución: materialización por demanda

En consecuencia, la solución es el patrón **Instanciación de Objetos Complejos** [Brown96], el cual propone aplazar la instanciación (materialización) de los objetos dependiendo de los patrones de acceso y de los requerimientos del desempeño. El caso extremo de este patrón —que se incluye en el presente capítulo para simplificar la exposición— es una materialización total por demanda: diferir la materialización de los objetos hasta que sea necesario. En muchas aplicaciones se recomienda adoptar una solución intermedia, en que una jerarquía de composición se materializa uno o dos niveles de profundidad. Con un intermediario distinto para cada objeto persistente, es posible decidir, intermediario por intermediario, el grado de materialización de los objetos persistentes y de sus objetos asociados.

38.17.3 Ejemplo: materialización de una instancia *VentasLineadeProducto*

Para entender la complejidad con que funciona la instanciación de objetos dentro del contexto del esquema persistente, examinaremos el caso de materializar la instancia *VentasLineadeProducto*. Supongamos que ella y su información relacionada se guardan en tablas relacionales, como se indica en la figura 38.16.

VENTASLINEADEPRODUCTO

IDO	cantidad
vli1	1
vli2	2

ESPECIFDEPRODUCTOS

IDO	descripcion	precio	cup
p1	pañuelos desechables	1.50	111
p2	tempeh	2.25	222

VENTASLINEADEPRODUCTO-A-ESPECIFDEPRODUCTOS

VLI-IDO	EP-IDO
vli1	p1
vli2	p2

Figura 38.16 Tabla de *VentasLineadeProducto* y su información relacionada.

Supongamos además que sabemos que el identificador de objetos de *VentasLineadeProducto* es “vli1”.¹ Veamos lo que ocurre si ejecutamos el siguiente código:

```
//crear el agente
AgenteVentasLineadeProducto unVLI =
    new AgenteVentasLineadeProducto("vli1"),

//causa materializacion de los objetos
int total = unVLI.subtotal();
```

La colaboración se muestra de manera gráfica a partir de la figura 38.17. Obsérvese que la proposición simple *unVLI.subtotal()* causa una importante materialización de objetos que no se observa directamente. Un paso decisivo se encuentra dentro del método *RegistroactualComoObjeto* de *IntermediarioRelacionaldeVentasLineadeProducto*. Una vez creada la instancia *VentasLineadeProducto*, el identificador de objetos de la *EspecificaciondeProductos* asociada se determina leyendo en la tabla asociativa.² En vez de materializar *EspecificaciondeProductos*, creamos un *AgentedeEspecificaciondeProductos* y lo asociamos a *VentasLineadeProducto*.

¹ Más adelante veremos cómo se determina el IDO del objeto inicializado para efectuar la materialización.
² Se emplea un seudo SQL para indicar la lectura a partir de otra tabla. Varían mucho los detalles de cómo se realiza esa lectura.

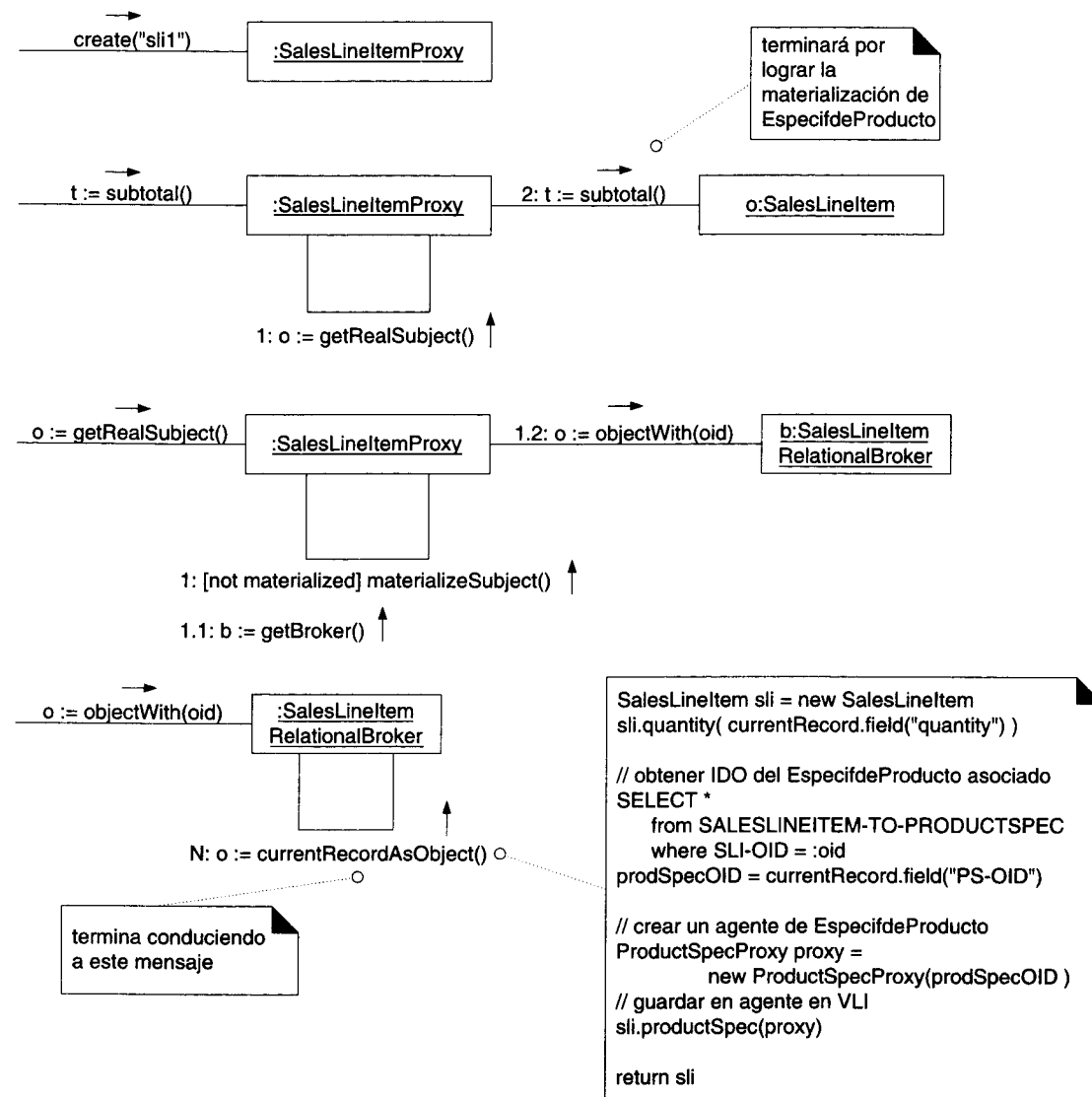


Figura 38.17 Materialización a partir de una instancia VentasLineadeProducto.

Así pues, *VentasLineadeProducto* referencia a un agente, no a la *EspecificaciondeProductos* real. Esta última no se materializará mientras no se le envíe el mensaje *precio*, como se advierte en la figura 38.18.

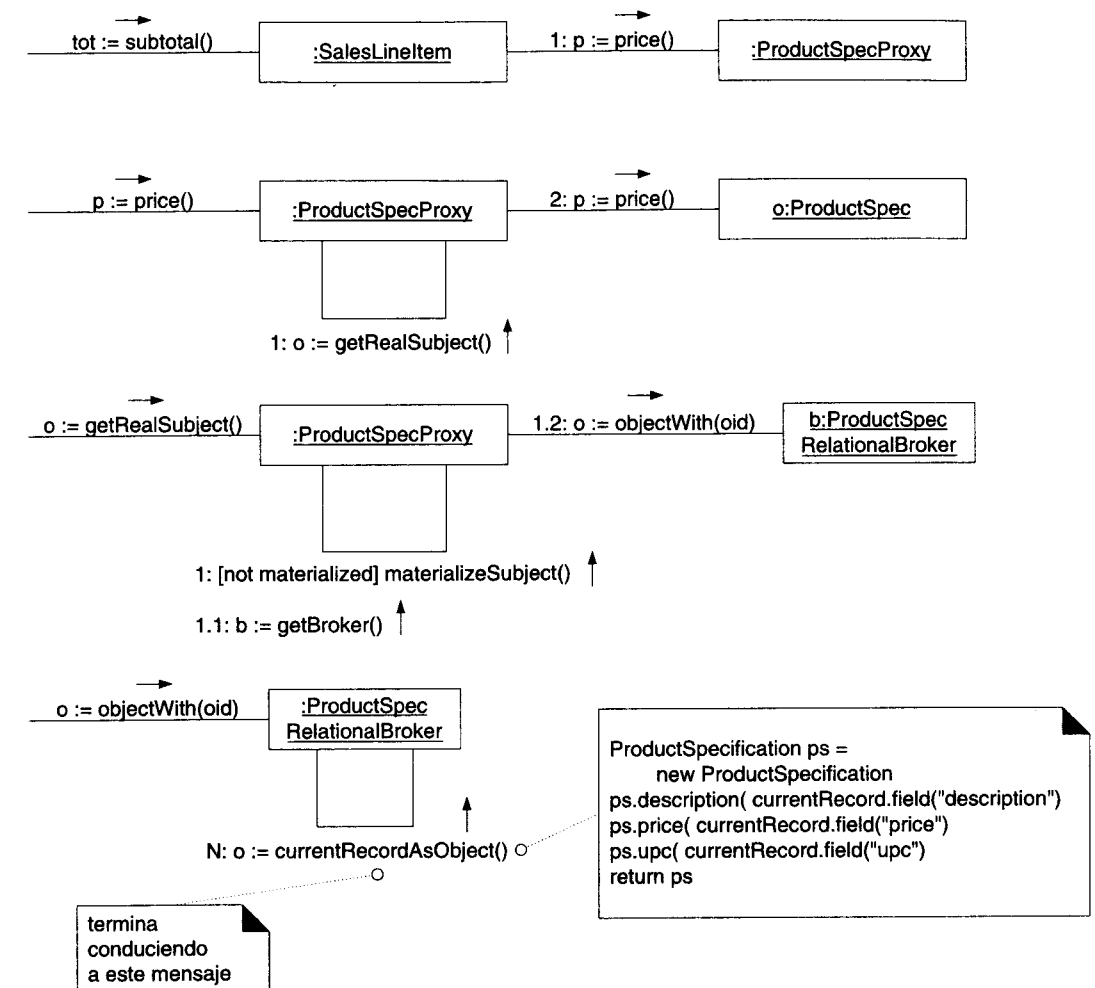


Figura 38.18 Materialización de una instancia EspecificaciondeProducto.

38.18 Operaciones de transacciones

Por lo general, una transacción acaba por ser guardada en una base de datos. Cuando se invoca una operación de *commit*, a los objetos se les trata de modo diferente según el estado de transacción que presenten. Por ejemplo, si hemos materializado un objeto viejo a partir de una base de datos pero sin modificarlos, no es necesario volver a introducirlo en la base. Por el contrario, si lo modificamos, habrá que actualizar el registro correspondiente de la base de datos.

38.18.1 Estado de transacción de los objetos

He aquí los posibles estados relevantes de la transacción:

1. **Limpio y nuevo.** Objetos nuevos, sin modificaciones.
2. **Limpio y viejo.** Objetos viejos materializados a partir de una base de datos, sin modificaciones.
3. **Sucio y nuevo.** Objetos nuevos, sin modificaciones.
4. **Sucio y viejo.** Objetos viejos materializados a partir de una base de datos, con modificaciones.
5. **Eliminar nuevo.** Objetos nuevos que deben ser eliminados.
6. **Eliminar viejo.** Objetos viejos que fueron materializados a partir de una base de datos y que deben ser eliminados.

Como vimos con anterioridad, un Intermediario de Base de Datos conservará cachés especiales para cada uno de estos estados y garantizará con ello que un objeto está en la caché apropiada.

Cuando un objeto se materializa por primera vez, su intermediario se encarga de colocarlo en la caché Limpio y Viejo. Cuando se modifica, se le cambia de sitio y se le pone en la caché Sucio y Viejo. Los objetos nuevos comienzan en la caché Limpio y Nuevo, pudiendo dirigirse después a la caché Sucio y Nuevo cuando sean modificados. Si se marca un objeto para eliminación, se le colocará en su respectiva caché Eliminar.

38.18.2 Cómo se ensucia un objeto

Un objeto se *ensucia* cuando modificamos uno de sus atributos. La forma más frecuente de señalar que se haga esto son los métodos mutador (establecedor). Por ejemplo:

```
// clase EspecificaciondeProducto
void price(float p)
{
    price = p;
    BrokerServer.instance().dirty(this);
}
```

A una Fabricación Pura *ServidordeIntermediario* del patrón Singleton y Fachada se le notificará que un objeto en particular está sucio. El *ServidordeIntermediario* encontrará el Intermediario apropiado de la Base de Datos para esta clase de objetos y le notificará que el objeto está sucio. El Intermediario entonces introduce el objeto en una caché sucia. Si era un objeto materializado, entrará en la caché Vieja y Sucia. Si era un objeto nuevo que todavía no se encuentre en la base de datos, se dirigirá hacia la Caché Nueva y Sucia.

¿Por qué enviar mensajes al objeto Fachada *ServidordeIntermediario* y no a cualquier otro? Porque así se conserva en un nivel mínimo el acoplamiento de los objetos del dominio. Él tan sólo sabe que una clase forma parte del esquema de persistencia: el *ServidordeIntermediario*. Con ello se da soporte al patrón Bajo Acoplamiento y se reduce al mínimo el impacto de los cambios.

38.18.3 Cómo eliminar

Si se desea eliminar un objeto, habrá que registrar explícitamente el hecho para que podamos realizar la modificación correspondiente en la base de datos después de una operación *commit*. Como en el caso de la señal *sucia*, esto se logra enviando un mensaje a *ServidordeIntermediario*. Por ejemplo:

```
// clase CatalogodeProductos
void removeProductSpec(ProductSpec p)
{
    // eliminar p en la colección de ProductSpecs
    productSpecs.remove(p);

    // notificarle al intermediario que p debe eliminarse
    en la base de datos
    BrokerServer.instance().delete(p);
}
```

A un *ServidordeIntermediario* del patrón Fachada Singleton puede notificársele que un objeto está sucio. El *ServidordeIntermediario* encontrará al intermediario de bases de datos correspondiente de esta clase de objeto y le notificará que el objeto está sucio. El intermediario de la base de datos introducirá entonces el objeto en una caché sucia. Si era un objeto materializado, los introduciremos en la Caché Vieja y Sucia. Si era un objeto nuevo que todavía no estaba en la base de datos, se dirigirá hacia la Caché Nueva y Sucia.

38.18.4 La operación commit

Cuando se decidió instalar la transacción, se envió un mensaje *commit* a la Fachada *ServidordeIntermediario*. Aunque cualquier objeto puede dirigir este mensaje, se acostumbra que un Coordinador de Aplicaciones se encargue de ello.

```
BrokerServer.instance().commit();
```

El método *ServidordeAgente-commit* simplemente dirige un mensaje *commit* a cada intermediario.

```
void BrokerServer.commit()
{
    for each broker b
        b.commit()
}
```

En el tiempo de una transacción los objetos pueden ser modificados, creados y eliminados. Suponiendo que se encuentren en la caché del estado correspondiente de la transacción (Viejo y Sucio, por ejemplo), he aquí las reglas de lo que deberá hacer el mensaje commit:

- Caché Nueva y Limpia.
 - insertar en base de datos.
 - dirigirse a Caché Vieja y Limpia.

- Caché Vieja y Limpia.
 - ignorar; no han cambiado.
- Caché Nueva y Sucia.
 - insertar en base de datos.
 - dirigirse a Caché Vieja y Limpia.
- Caché Vieja y Sucia.
 - actualizar en base de datos.
 - dirigirse a Caché Vieja y Limpia.
- Caché Nueva Eliminada.
 - eliminar en caché.
- Caché Vieja Eliminada.
 - eliminada en la base de datos.
 - Eliminar en caché.

38.18.5 La operación de rollback

Una vez que se haya decidido someter la transacción a un rollback, se envía un mensaje *rollback* a la Fachada *ServidordeIntermediario*. Aunque cualquier objeto puede hacerlo, se acostumbra que el Coordinador de Aplicaciones asuma esta responsabilidad.

```
BrokerServer.instance().rollback();
```

El método *ServidordeIntermediario-rollback* simplemente envía un mensaje de *rollback* a cada intermediario.

He aquí las reglas de lo que debe hacer una instalación:

- Caché Vieja y Limpia.
 - ignorar; no han cambiado.
- El resto de las cachés.
 - eliminar en la caché.

Una de las ventajas del uso constante de Agente Virtual con un objeto y con una referencia directa es el efecto que se obtiene en el caso de un rollback. Una vez vaciadas las cachés, todos los Agentes Virtuales se referirán a los objetos no materializados y no a los modificados en la memoria local. La siguiente referencia a un Agente Virtual hará que el objeto viejo sea rematerializado a partir del almacenamiento persistente; cualquier cambio del objeto se evaporó al vaciar la caché.

38.19 Búsqueda de objetos en el almacenamiento persistente

¿Cómo podemos recuperar un registro a partir de un almacenamiento persistente? La solución depende de las herramientas, de las bibliotecas y el ambiente operativo. Por ejemplo, en los ambientes del sistema operativo de Microsoft, podemos valernos de los servicios DAO dentro de MFC. Rebasa el ámbito de nuestra exposición ofrecer más detalles.

Dentro del esquema hemos definido el método *Intermediario de EPRelacional-SeleccionarPrimero(consulta)* para prestar el servicio de localizar el primer registro (renglón de una tabla) que reúna los criterios de la consulta. Su implementación depende del ambiente operativo.

Un Intermediario de Base de Datos necesita ofrecer dos patrones de búsqueda cuando se recuperen registros a partir de un almacenamiento persistente:

- Búsqueda mediante el identificador de objetos (el criterio más común de búsqueda).
- Búsqueda mediante criterios arbitrarios, como la clave primaria del dominio (por ejemplo, el número del seguro social).

Cuando se utiliza un esquema de persistencia, enfrentamos el problema de *preparar el escenario*: ¿con qué criterios debería recuperarse el objeto raíz en una jerarquía de composición? Los objetos no raíz pueden materializarse utilizando Agentes Virtuales y realizando la búsqueda con sus identificadores de objeto como clave de consulta. Pero normalmente no conoceremos el identificador del objeto raíz. Consideremos, por ejemplo, el caso de materializar una instancia *Venta*, su instancia relacionada *VentasLineadeProducto* y sus *EspecificacionesdeProducto* asociadas. Suponiendo que se haya materializado la *Venta* y que tuviera referencias a Agentes Virtuales para todas sus *VentasLineadeProducto*, las líneas de productos se materializarán por demanda, basándose en los valores del identificador de objetos en los agentes. De manera análoga, si se materializó una instancia *VentasLineadeProducto* y si tenía una referencia a un Agente Virtual para su *EspecificaciondeProducto*, también podemos materializarla por medio de su valor de identificador de objetos. Pero si en esta jerarquía de composición no se conoce el identificador del objeto raíz *Venta*, ¿cómo preparamos la escena e introduciremos en la memoria esta primera instancia *Venta*?

El problema anterior nos indica la necesidad de contar con capacidades de búsqueda orientadas al dominio, entre ellas la búsqueda de *Ventas* mediante su fecha y hora o alguna otra clave primaria adecuada. Así, en el caso de la *Venta*, la clase *Intermediario-RelacionaldeVentas* puede ofrecer además un método público para efectuar la materialización por fecha y hora, como se advierte en la figura 38.19.

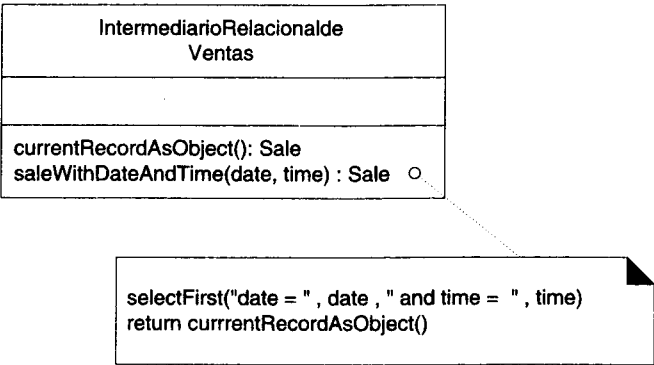


Figura 38.19 Capacidades de búsqueda orientadas al dominio.

38.20 Diseños alternos

38.20.1 Metadatos e intermediarios parametrizados

En contraste con la creación de un Intermediario de Base de Datos concreto e independiente para cada clase de objetos persistentes, podemos definir metadatos (datos acerca de datos) respecto al mapeo de clases y tablas, respecto al mapeo de los nombres de atributos y campos, etc. Los metadatos pueden conservarse en un objeto *MetadatosdeAlmacenamientoPersistente*. En este enfoque no es necesario elaborar una jerarquía extensa de intermediarios. Es posible, por ejemplo, que el *Intermediario-deEPRelacional* sea una clase concreta instanciada y parametrizada con metadatos para cada clase de objetos persistentes. No se requieren subclases de *Intermediario-deEPRelacional*.

El enfoque de metadatos es más robusto y flexible que el de formación de subclases adoptado en el actual esquema persistente; se recomienda en aplicaciones que contengan muchas clases de objetos persistentes y esquemas rápidamente cambiantes. Por ser más complejos de explicar e implementar, no lo estudiaremos a fondo. Si el lector desea más información, le recomendamos consultar el patrón Reflexión en [BMRSS96].

38.20.2 Objetos consulta

A diferencia de las consultas de cadenas simples que se ejemplifican en esta exposición (por ejemplo, "OID = 123"), es posible crear una clase *Consulta* cuyas instancias estén parametrizadas con expresiones booleanas. Este esquema tiene la ventaja de abstraer de cualquier lenguaje de manipulación de datos (SQL, por ejemplo) y de manipular las consultas y razonar con ellas (por ejemplo, compilarlas para darles una forma más eficiente).

38.20.3 Cambio de intermediarios y de intermediarios de bases de datos en la memoria

Es posible y conveniente crear un Intermediario en-la Memoria que no guarde objetos en un almacenamiento persistente externo cuando se envía la señal de *commit*. Esto es muy útil al iniciarse el desarrollo y las pruebas, pues se evita el impacto que en el desempeño tiene el utilizar un intermediario real y la complejidad de su implementación completa.

Los clientes de todas las subclases de *IntermediariodeEP* emplean sólo unos cuantos métodos públicos, entre ellos *objetoCon(unIDO)*; podemos desconectar un intermediario y conectar otro, sin que ello afecte a los objetos cliente. Así, un diseñador puede utilizar un Intermediario en-la Memoria (figura 38.20) durante algún periodo y luego remplazarlo por un intermediario relacional o plano. El cambio puede efectuarse modificando el Método Fábrica *crearIntermediario* en el patrón Agente Virtual, que especifica cuál intermediario emplear.

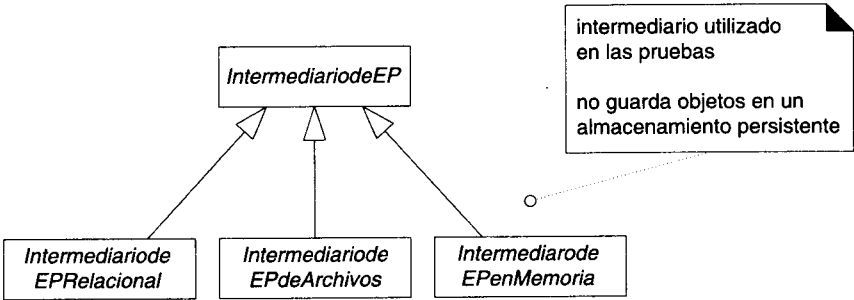


Figura 38.20 Intermediario en-la Memoria.

38.20.4 Comparación entre estados de transacción y cachés múltiples

Colocar un objeto en una de las cachés de intermediarios, digamos la caché *ViejaLimpia* o *ViejaSucia*, es una forma de recordar el estado de transacción del objeto. En el caso de un intermediario, se trata de un mecanismo eficiente. Pero podemos recurrir a un diseño alternativo donde cada objeto se asocie a un objeto *Estado de Transacción* que indica si es viejo y limpio, viejo y sucio, etc. Todos los objetos pueden estar en una caché, y el estado de transacción del objeto se conoce mediante el objeto asociado de estado y no mediante su pertenencia a la caché.

¿Dónde debería recordarse la relación *Estado de Transacción*? Conviene no agregar directamente el conocimiento de la persistencia a las definiciones del objeto del dominio; de ahí que recomendamos las siguientes opciones:

- Si se usa una superclase común *ObjetoPersistente*, el estado es un atributo definido en esa clase.
- En un *Mapa* (*Dictionary* o *Hashtable*) que conserva el intermediario. La clave del *Mapa* es el objeto, el valor asociado *Mapa* es el *Estado de Transacción* del objeto.

- Como un atributo del *AgenteVirtual*. Este diseño presenta una complicación: si varios agentes se relacionan con el mismo objeto real, todos ellos habrán de permanecer sincronizados.

Además, cada clase *EstadoTransaccion* puede definir un método que especifique lo que ha de hacerse en un commit o rollback. Y como los objetos estado probablemente no conserven ninguna información, también pueden ser singletons: sólo una instancia de cada clase se necesita en la aplicación.

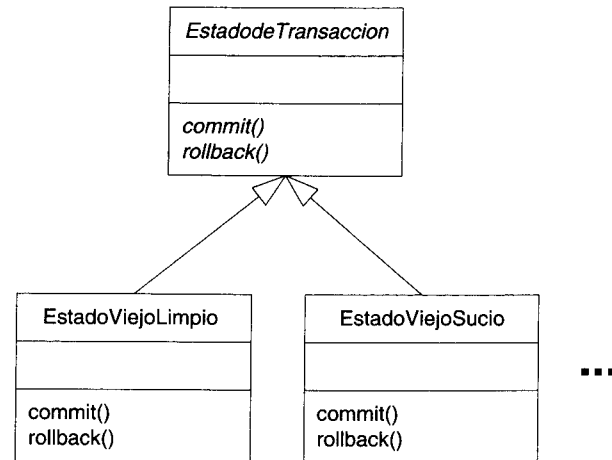


Figura 38.21 Estados de transacción de los objetos persistentes.

38.21 Cuestiones sin resolver

Hemos dado una brevisima introducción a los problemas y a las soluciones de diseño en un esquema de persistencia. Ofrecimos una sucinta exposición de varios temas importantes:

- Desmaterialización de objetos. En resumen, los agentes concretos deben definir un método *objetoComoRegistro* que transforme un objeto en registro. Para desmaterializar las jerarquías de composición se requieren la colaboración entre varios intermediarios y la conservación de tablas asociativas (si se emplea una base de datos relacional).
- Apuntadores hacia atrás. Un esquema de persistencia a veces tienta al diseñador para que a otros objetos les agregue apuntadores hacia atrás que el objeto no requeriría por sí mismo.
- Materialización y desmaterialización de colecciones.
- Manejo de errores cuando fracasa una operación de base de datos.
- Estrategias de acceso a multiusuarios y de bloqueo.
- Seguridad: control del acceso a la base de datos.

APÉNDICE A. LECTURAS RECOMENDADAS

Análisis y diseño generales orientados a objetos

Booch, G., 1994. *Object-Oriented Analysis and Design*. Redwood City, CA.: Benjamin/Cummings.

Coad, P. 1995. *Object Models: Strategies, Patterns and Applications*. Englewood Cliffs, NJ.: Prentice-Hall.

Coleman, D., et al. 1994. *Object-Oriented Development: The Fusion Method*. Englewood Cliffs, NJ.: Prentice-Hall.

Jacobson, I., et al. 1992. *Object-Oriented Software Engineering: A Use Case Driven Approach*. Reading, MA.: Addison-Wesley.

Rumbaugh, J., et al. 1991. *Object-Oriented Modelling and Design*. Englewood Cliffs, NJ.: Prentice-Hall.

Construcción de modelos conceptuales

Fowler, M. 1996. *Analysis Patterns: Reusable Object Models*. Reading, MA.: Addison-Wesley.

Hay, D. 1996. *Data Model Patterns: Conventions of Thought*. NY, NY.: Dorset House.

Martin, J. y Odell, J. 1995. *Object-Oriented Methods: A Foundation*. Englewood Cliffs, NJ.: Prentice-Hall.

Diseño orientado a objetos

- Riel, A. 1996. *Object-Oriented Design Heuristics*. Reading, MA.: Addison-Wesley.
- Wirfs-Brock, R., Wilkerson, B. y Wiener, L. 1990. *Designing Object-Oriented Software*. Englewood Cliffs, NJ.: Prentice-Hall.

Patrones

- Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P. y Stal, M. 1996. *A System of Patterns*. West Sussex, Inglaterra: Wiley.
- Coad, P. 1995. *Object Models: Strategies, Patterns and Applications*. Englewood Cliffs, NJ.: Prentice-Hall.
- Fowler, M. 1996. *Analysis Patterns: Reusable Object Models*. Reading, MA.: Addison-Wesley.
- Gamma, E., Helm, R., Johnson R. y Vlissides, J. 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA.: Addison-Wesley.
- Riel, A. 1996. *Object-Oriented Design Heuristics*. Reading, MA.: Addison-Wesley.
- Varios editores. *Pattern Languages of Program Design*. Todos los volúmenes. Reading, MA.: Addison-Wesley.

Análisis de requerimientos

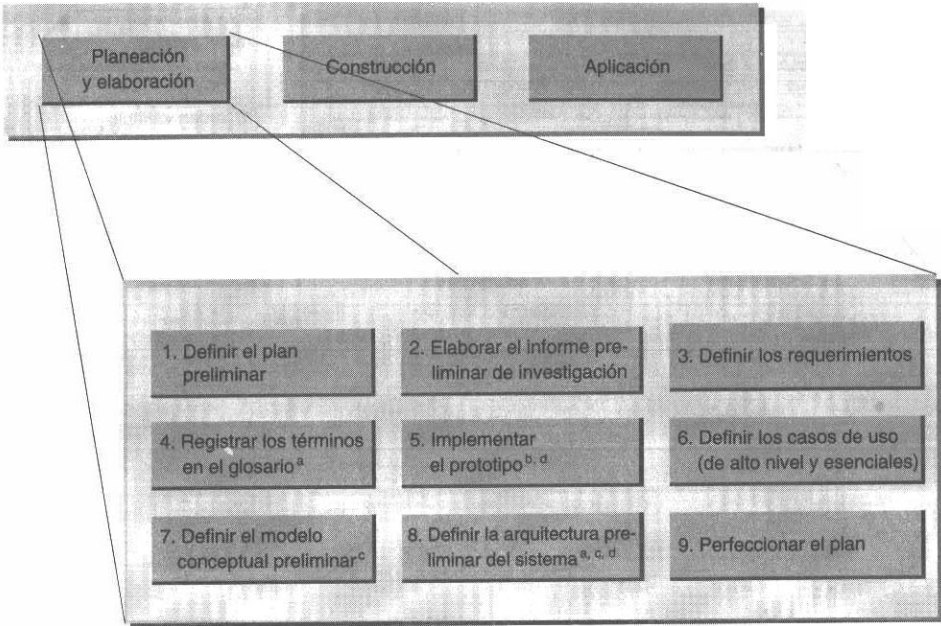
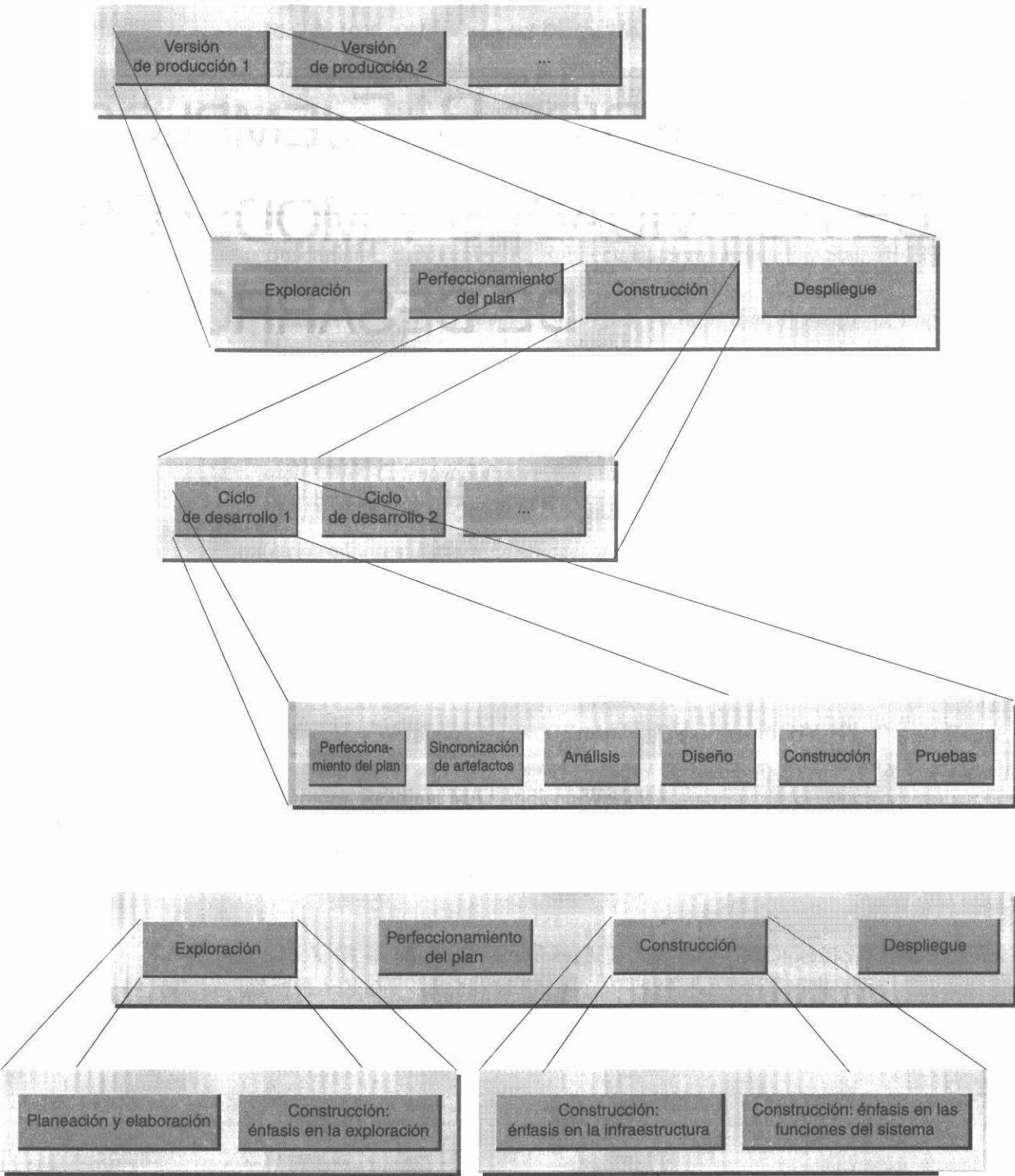
- Gause, D. y Weinberg, G. 1989. *Exploring Requirements*. NY, NY.: Dorset House.
- Jacobson, I., *et al.* 1992. *Object-Oriented Software Engineering: A Use Case Driven Approach*. Reading, MA.: Addison-Wesley.

El UML (Unified Modeling Language)

- Booch, G., Jacobson, I. y Rumbaugh, J. 1997. The UML specification documents. Santa Clara, CA.: Rational Software Corp. Available at www.rational.com.

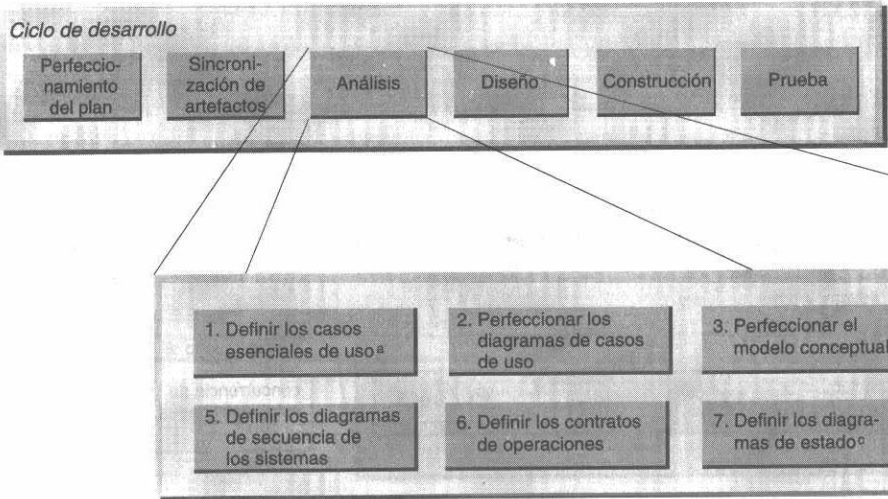
APÉNDICE B. EJEMPLOS DE ACTIVIDADES Y MODELOS DE DESARROLLO

Ejemplo de actividades de desarrollo

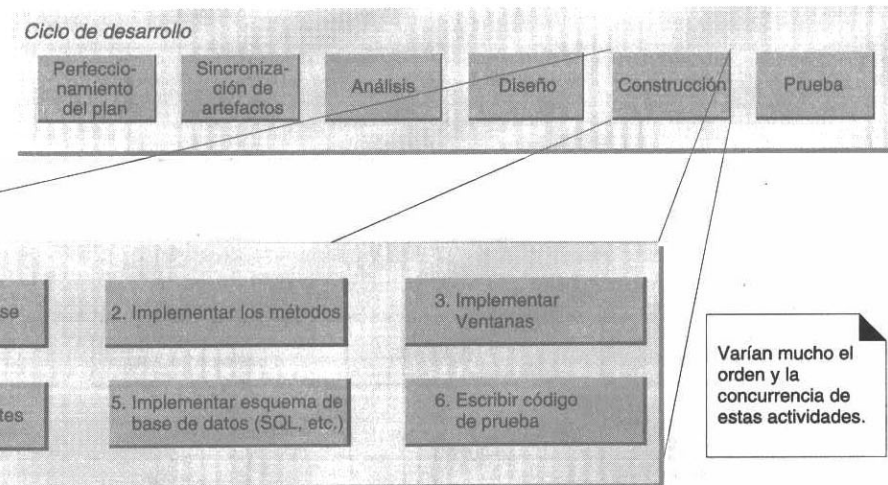
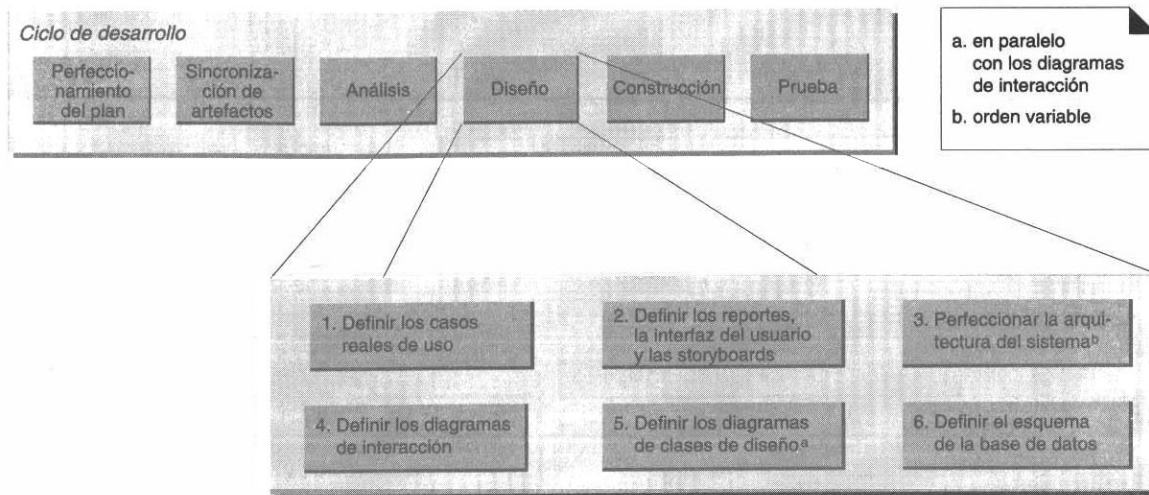


Notas

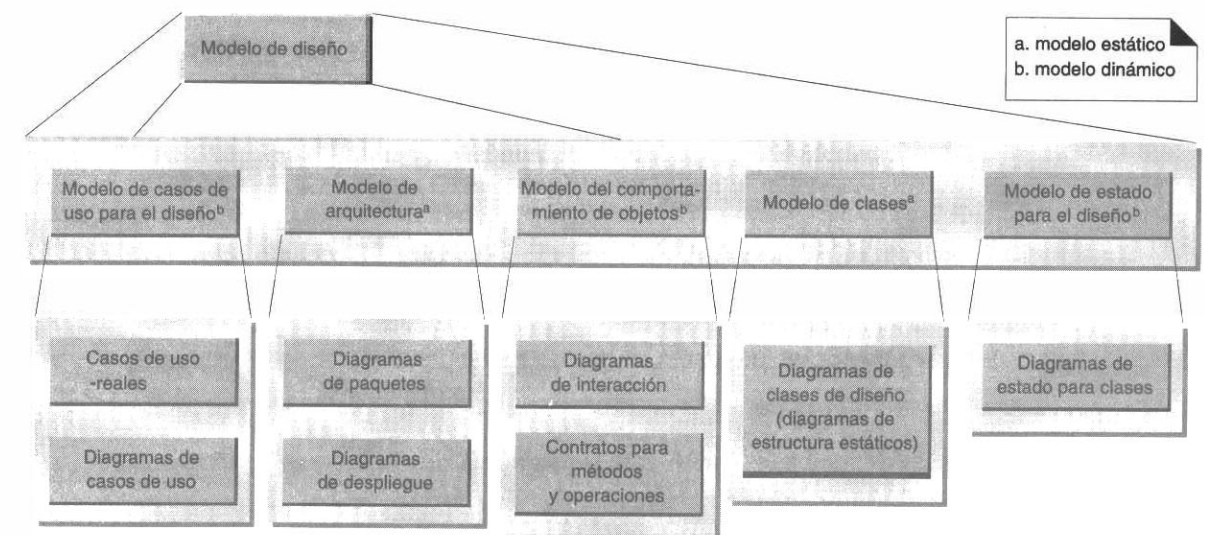
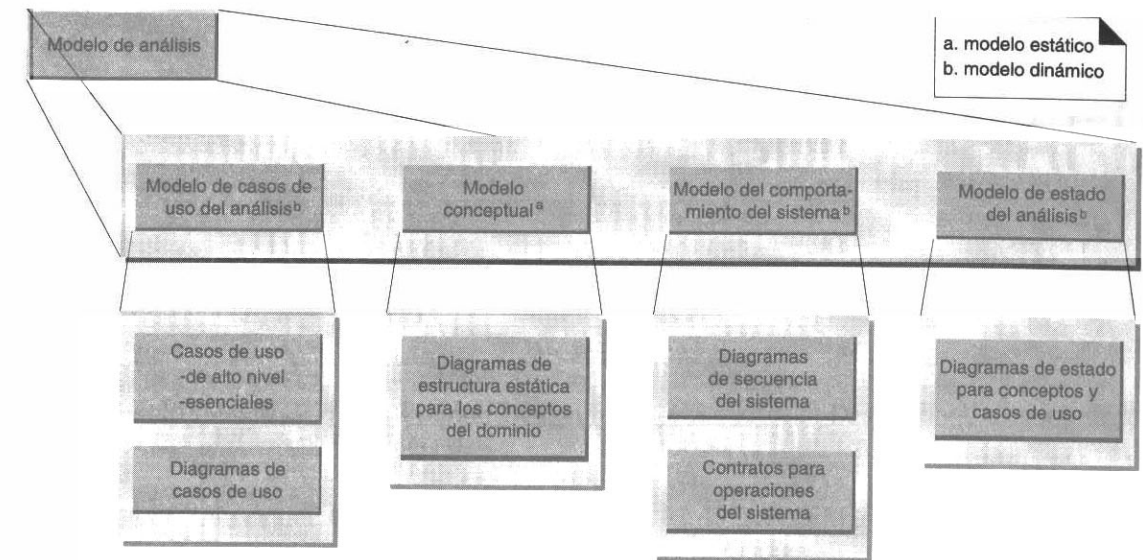
- a. constante
- b. opcional
- c. aplazable
- d. orden variable



a. si todavía no se realiza
b. actual
c. opcional



Ejemplos de modelos



BIBLIOGRAFÍA

- Abbot83** Abbott, R. 1983. Program Design by Informal English Descriptions. *Communications of the ACM* vol. 26(11).
- AlS77** Alexander, C., Ishikawa, S. y Silverstein, M. 1977. *A Pattern Language-Towns-Building-Construction*. Oxford University Press.
- BC89** Beck, K. y Cunningham, W. 1989. A Laboratory for Object-oriented Thinking. *Proceedings of OOPSLA 89*. SIGPLAN Notices, vol. 24, núm. 10.
- Beck94** Beck, K. 1994. Patterns and Software Development. *Dr Dobbs Journal*. Feb., 1994.
- BJR97** Booch, G., Jacobson, I. y Rumbaugh, J. 1997. The UML specification documents. Santa Clara, CA.: Rational Software Corp. Véanse los documentos en www.rational.com.
- BMRSS96** Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P. y Stal, M. 1996. *A System of Patterns*. West Sussex, Inglaterra: Wiley.
- Booch94** Booch, G., 1994. *Object-Oriented analysis and Design*. Redwood City CA.: Benjamin/Cummings.
- Booch96** Booch, G., 1996. *Object Solutions: Managing the Object-Oriented Project*. Menlo Park, CA.: Addison-Wesley.
- Brooks75** Brooks, F., 1975. *The Mythical Man-Month*. Reading, MA.: Addison-Wesley.
- Brown96** Brown, K. y Whitenack, B. 1996. Crossing Chasms. *Pattern Languages of Program Design* vol. 2. Reading, MA.: Addison-Wesley.
- Coad95** Coad, P. 1995. *Object Models: Strategies Patterns and Applications*. Englewood Cliffs, NJ.: Prentice-Hall.

Coleman94 Coleman, D., et al. 1994. *Object-Oriented Development: The Fusion Method*. Englewood Cliffs, NJ.: Prentice-Hall.

Constantine97 Constantine, L. 1997. The Case for Essential Use Cases. *Object Magazine*. Mayo, 1997. NY, NY: SIGS Publications.

Coplien95 Coplien, J. 1995. *The History of Patterns*. Véase <http://c2.com/cgi/wiki?HistoryOfPatterns>.

Fowler96 Fowler, M. 1996. *Analysis Patterns: Reusable Object Models*. Reading, MA.: Addison-Wesley.

Gartner95 Schulte, R., 1995. *Three-Tier Computing Architectures and Beyond*. Informe Publicado. GartnerGroup.

GHJV95 Gamma, E., Helm, R., Johnson, R. y Vlissides, J. 1995. *Design Patterns*. Reading, MA.: Addison-Wesley.

GW89 Gause, D. y Weinberg, G. 1989. *Exploring Requirements*. NY, NY.: Dorset House.

Jacobson92 Jacobson, I., et al. 1992. *Object-Oriented Software Engineering: A Use Case Driven Approach*. Reading, MA.: Addison-Wesley.

Lieberherr88 Lieberherr, K., Holland I. y Riel, A. 1988. Object-Oriented Programming: An Objective Sense of Style. *OOPSLA 88 Conference Proceedings*. NY, NY: ACM SIGPLAN.

Mo95 Martin, J. y Odell, J. 1995. *Object-Oriented Methods: A Foundation*. Englewood Cliffs, NJ.: Prentice-Hall.

Rumbaugh91 Rumbaugh, J., et al. 1991. *Object-Oriented Modelling and Design*. Englewood Cliffs, NJ.: Prentice-Hall.

Rumbaugh97 Rumbaugh, J. 1997. Models Through the Development Process. *Journal of Object-Oriented Programming*, Mayo, 1997. NY NY: SIGS Publications.

Standish94 Anónimo. 1994. *Charting the Seas of Information Technology: Chaos*. Informe publicado. The Standish Group.

Wirfs-Brock93 Wirfs-Brock, R. 1993. Designing Scenarios: Making the Case for a Use Case Framework. *Smalltalk Report*, nov-dic 1993. NY, NY: SIGS Publications.

GLOSARIO

abstracción	Acción de concentrar las cualidades esenciales o generales de cosas similares. También, las características esenciales resultantes de una cosa.
acoplamiento	Dependencia entre elementos (generalmente tipos, clase y subsistemas), normalmente debida a la colaboración entre ellos para prestar un servicio.
agregación	Propiedad de una asociación que representa una relación todo-parte y donde (normalmente) se le contiene por toda la vida.
análisis	Investigación de un dominio, la cual da origen a modelos que describen sus características estáticas y dinámicas. Se centra en cuestiones de “qué” más que de “cómo”.
análisis orientados a objetos	Investigación del dominio o sistema de problemas a partir de los conceptos de dominio, como tipos de objetos, asociaciones y cambios de estado.
arquitectura	Descripción de la organización y estructura de un sistema. Varios niveles de arquitecturas intervienen en la creación de sistemas de software, desde la arquitectura física del hardware hasta la arquitectura lógica de un esquema de aplicaciones.
asociación	Descripción de un conjunto relacionado de enlaces o vínculos entre objetos de dos tipos.
asociación calificada	Aquella cuya pertenencia parte o divide el valor de un calificador.
asociación recursiva	Aquella en que la fuente y el destino son el mismo tipo de objeto.
atributo	Característica o propiedad de tipo con un nombre asignado.
atributo de clase	Característica o propiedad que es igual en todas las instancias de una clase. Esta información suele almacenarse en la definición de la clase.
caso de uso	Descripción narrativa textual de la secuencia de eventos y acciones que ocurren cuando un usuario parte o divide en un diálogo con un sistema durante un proceso significativo.

clase	En el lenguaje UML, “descripción de un conjunto de objetos que comparten los mismos atributos, operaciones, métodos, relaciones y significado” [BJR97]. En este libro, a menudo se usa como sinónimo de una “clase de implementación” de UML: el mecanismo de software con que se definen e implementan los atributos y métodos de un tipo en particular.
clase abstracta	Clase que puede emplearse exclusivamente como superclase de alguna otra clase; no es posible crear objetos de una clase abstracta, salvo como instancias de una subclase. Una clase abstracta suele servir para definir una interfaz común de varias subclases.
clase concreta	Aquella que puede tener instancias.
clase de contenedor	Aquella diseñada para alojar y manipular una colección de objetos.
clasificación	La que define una relación entre un tipo y sus instancias. El mapeo taxonómico identifica la extensión de un tipo.
colaboración	Dos o más objetos que participan en la relación de cliente/servidor, con el propósito de prestar un servicio.
componente	Módulo discreto de software con una interfaz.
composición	La identificación de un tipo en que cada instancia está constituida por otros objetos.
concepto	Categoría de ideas o cosas. En este libro, con él se designan cosas del mundo real y no entidades de software. La intensión de un concepto es la descripción de sus atributos, operaciones y significado. La extensión de un concepto es el conjunto de instancias u objetos muestra que pertenecen a él. A menudo se define como sinónimo de tipo.
conformidad	Relación entre dos tipos tal que, si el tipo X se conforma al tipo Y, los valores de aquél son también miembros de éste y satisfacen la definición del tipo Y.
constructor	Método especial que se invoca siempre que la instancia de una clase se crea en el lenguaje C++ o en el lenguaje Java. El constructor a menudo lleva a cabo acciones de inicialización.
contrato	Define las responsabilidades y poscondiciones que se aplican al uso de una operación o método. También designa el conjunto de las condiciones relacionadas con una interfaz.
delegación	Idea de que un objeto puede enviarle un mensaje a otro en respuesta a un mensaje. Por tanto, el primer objeto delega la responsabilidad al segundo.
derivación	Proceso que consiste en definir una nueva clase por referencia a otra existente y agregar luego atributos y métodos. La clase existente es la superclase; a la nueva se le llama subclase o clase derivada.
descomposición funcional	Proceso de refinar la solución de un problema descomponiéndolo varias veces en pasos funcionales cada vez más pequeños.

diagrama de estado	Forma de una máquina de estado finito con que se describe el comportamiento dinámico de un tipo.
diseño	Proceso que se sirve de los productos del análisis para generar una especificación destinada a implementar un sistema. Descripción lógica de cómo funcionará un sistema.
diseño orientado a objetos	Especificación de una solución lógica de software a partir de objetos de software: clases, atributos, métodos y colaboraciones.
dominio	Límite formal que define determinado tema o área de interés.
encapsulamiento	Mecanismo con que se ocultan los datos, la estructura interna y los detalles de la implementación de un objeto. La interacción con un objeto se realiza a través de una interfaz pública de las operaciones.
especificaciones de requerimientos	Documento que describe lo que hace un sistema de software: sus funciones y sus atributos. Generalmente escritas desde el punto de vista del usuario.
esquema	Conjunto de clases abstractas y concretas que colaboran y que pueden servir de plantilla para resolver una familia afín de problemas. Por lo regular se extendía formando subclases del comportamiento propio de la aplicación.
estado	Condición de un objeto entre eventos.
evento	Ocurrencia notable.
extensión	Conjunto de objetos a los que se aplica un concepto. Los objetos de la extensión son ejemplos o instancias del concepto.
generalización	Actividad consistente en identificar aspectos comunes entre conceptos y en definir las relaciones entre el supertipo (concepto general) y el subtipo (concepto especializado). Es una forma de hacer clasificaciones taxonómicas entre conceptos que luego se explican con ejemplos en las jerarquías de tipos. Los subtipos se conforman a los supertipos en cuanto a la intensión y la extensión.
herencia	Característica de los lenguajes de programación orientados a objetos, en virtud de la cual las clases pueden especializarse a partir de superclases más generales. La subclase adquiere automáticamente las definiciones de atributos y clases hechas a partir de las superclases.
identidad del objeto	Característica de que la existencia de un objeto es independiente de los valores asociados a él.
IDO	Identificador del objeto.
instancia	Miembro individual de un tipo o de una clase.
instanciación	Creación de una instancia de un tipo o de una clase.

intensión	Definición de un concepto.
interfaz	Conjunto de representaciones de operaciones públicas.
jerarquía de clase	Descripción de las relaciones de herencia entre clases.
lenguaje de programación orientado a objetos	Aquel que soporta los conceptos de encapsulamiento, herencia y polimorfismo.
mensaje	Mecanismo en virtud del cual los objetos se comunican entre sí; generalmente una respuesta para ejecutar un método.
metamodelo	Modelo que define otros modelos. El metamodelo de UML define los tipos de elementos del lenguaje; por ejemplo Tipo y Operación.
método	En el lenguaje UML, implementación o algoritmo específicos de la operación de una clase. Informalmente, procedimiento de software que puede ejecutarse en respuesta a un mensaje.
método de clase	Aquel que define el comportamiento de una clase en contraste con el de sus instancias.
método de instancia	Aquel cuyo ámbito es una instancia. Se invoca enviando un mensaje a una instancia.
modelo	Descripción de las características estáticas, dinámicas o ambas de un tema, presentada en varias vistas (generalmente digramáticas o textuales).
multiplicidad	Número de objetos a los que se permite participar en una asociación.
objeto	En el lenguaje UML, instancia de una clase que encapsula el estado y el comportamiento. Más informalmente, ejemplo de una cosa.
objeto activo	Aquel que tiene su propia red de control.
objeto persistente	Aquel que puede sobrevivir al proceso o el hilo que lo creó. Un objeto persistente existe mientras no sea eliminado explícitamente.
operación	En el lenguaje UML, “servicio que puede solicitarse a un objeto para realizar el comportamiento” [BJR97]. Una operación tiene una representación, especificada por su nombre y parámetros, y se la invoca a través de un mensaje. Un método es una implementación de una operación mediante un algoritmo específico.
operación polimórfica	La misma operación implementada de modo distinto por dos o más tipos.
papel	Fin nombrado de una asociación que indica su propósito.

patrón	Es la descripción etiquetada de un problema, de la solución, de cuándo aplicar la solución y la manera de hacerlo dentro de otros contextos.
persistencia	Almacenamiento duradero del estado de un objeto.
polimorfismo	Concepto según el cual dos o más tipos de objetos pueden responder a un mismo mensaje en formas diferentes, usando para ello operaciones polimórficas. También, capacidad de definir las operaciones polimórficas.
poscondición	Restricción que debe cumplirse una vez terminada una operación.
precondición	Restricción que debe cumplirse antes que se solicite una operación.
privado	Mecanismo de ámbito con el cual se limita el acceso a los miembros de una clase para que otros objetos no puedan verlos. En condiciones normales se aplica a todos los atributos y a algunos métodos.
público	Mecanismo de ámbito con el cual se hace a los miembros accesibles a otros objetos. Normalmente se aplica a algunos métodos pero no a los atributos, porque los atributos públicos violan el encapsulamiento.
receptor	Objeto al cual se envía un mensaje.
responsabilidad	Servicio o grupo de servicios ofrecidos por un tipo; una responsabilidad abarca uno o más propósitos u obligaciones de un tipo.
restricción	Limitación o condición que se impone a un elemento.
subclase	Especialización de otra clase (la superclase). Una subclase hereda los atributos y métodos de la superclase.
subtipo	Especialización de otro tipo (el supertipo) que se conforma a la connotación y denotación del supertipo.
superclase	Clase cuyos atributos y métodos hereda otra clase.
supertipo	En una relación de generalización-especialización, el tipo más general; objeto que tiene subtipos.
tipo	En el lenguaje UML, descripción de un conjunto de objetos similares con atributos y operaciones, pero que no incluye métodos. Algunos autores definen el tipo y el concepto como sinónimos.
tipo abstracto	Tipo tal que todos los objetos que se conforman a él deben conformarse también a uno de sus subtipos.
tipo concreto	Aquel que puede tener instancias.

transición	Relación entre estados que se cruzan, si el evento especificado ocurre y se cumple la condición de custodia.
transición de estado	Cambio de estado en un objeto; algo que puede indicarse con un evento.
valores puros de datos	Tipos de datos para los cuales la identidad única de instancia no es significativa; por ejemplo, números, booleanos y cadenas. En el lenguaje UML, también se conocen como Tipos de Datos.
variable de instancia	Usado en Java y en Smalltalk, atributo de una instancia.
vínculo	Conexión entre dos objetos; instancia de una asociación.
visibilidad	Capacidad de ver o tener referencia a un objeto.

A

acoplamiento, 200
actor, 52
iniciador, 52
participante, 53
adecuación, 420
Agente (Proxy)
dispositivo, 400
remoto, 415
virtual, 468
agregación, 198, 359
compartida, 360
compuesta, 359
agregación de compuestos, 359
Alta Cohesión, 203
Alto nivel, caso de uso de, 56
análisis, 6
análisis de requerimientos, 8
análisis del dominio orientado a objetos, 8
análisis estructurado, 90
análisis orientado a objetos, 6
análisis y diseño
definición de, 6
análisis y diseño estructurados, 14
análisis y diseño, modelos de, 29-30
análisis y diseño orientados a objetos, 6
analogía de, 7
definición de, 6
ejemplo del juego de dados, 10
principios básicos de, 3-16
y descomposición funcional, 14
arquitectura, 437
arquitectura de tres capas, 273
arquitectura multicapas, 274
artefacto, 30
relaciones, 31
asignación de responsabilidades, 9
importancia de, 5
asociación, 105
adición de, 105-108
asignación de nombre, 111
calificada, 365
criterios de, útil, 106
de alta prioridad, 109
destacar la necesidad de conocer, 115
directrices de, 110
múltiple entre tipos, 112
multiplicidad de, 110
navegabilidad, 264
nivel de detalle, 109
nombres de papeles, 362
notación de UML, 106
obtención con lista de, 107
recursiva, 366

reflexiva, 366
vínculo, 173
atributo, 120
adición, 119-130
de referencia, 300
derivado, 128, 364
notación UML, 120
simple, 121
sin claves foráneas, 123
tipos no primitivos, 124
tipos válidos, 120
valor puro de datos, 121, 22
y cantidades, 125
atributos sistémicos, 42

B

Bajo acoplamiento, 200

C

calificador, 365
capa, 349
caso de uso, 8, 10, 49
abstracto, 53
asignación de nombre, 61
clasificación, 76
clasificación y programación, 73-80
creación de, 47-71
creación de, real, 163-166
cuándo crear casos abstractos de uso, 321
de alto nivel, 26, 56
de ejemplo real, 165
diagrama de, 55
diagrama de estado, 382
ejemplo de relaciones de usos, 323
error común en, 53
esencial, 58
esencial comparado con real, 58
expandido, 27, 50, 56
formatos de, 55
funciones y rastreabilidad de sistemas, 55
identificación de, 53
Iniciar, 76
límites de sistemas, 56
primario, 58
proceso de, 54
programación, 75
real, 59, 163
relación, 321-328
relaciones de usos, 322
secundario, 58
versiones, 77

y proceso de desarrollo, 63
y procesos del dominio, 54
casos de desarrollo del sistema, 450
ciclo de desarrollo
 inicio del, 81-82
 programación de casos de uso, 75
 segundo ciclo, 317-319
clase, 102
 abstracta, 347
 notación UML, 257
código
 creación de, 295-308
 solución de, en Java, 309-313
cohesión, 203
 baja, 210
Comando, 421
Comando, patrón de, 214
comportamiento
 del sistema, 137, 147
comportamiento sistémico, 137, 147
 principios básicos del, 137
compuesto, 359
concepto(s), 89
 comparados con papel, 363
 comparados con tipos e interfaces, 101
 conceptos semejantes, 97
 error en la obtención de, 97
 especificación o descripción de, 99
 extensión de, 89
 intensión de, 89
 notación UML, 87
 obtención de, con identificación de sustantivos, 93
 obtención de, con una lista de categorías, 91
 símbolo de, 89
 Véase también tipo
construcción, 6
contrato, 147
 descripción de secciones del, 148
 directrices y, 154
 ejemplo de, 147
 elaboración de, 145-158
 grado de terminación, 153
 poscondición del, 150
 precondiciones del, 153
 y descripción de algoritmos, 153
Controlador, 206
 aplicación, 223
 inflado, 210
coordinador de aplicaciones, 287
CRC, 215
Creador, 197, 225
 aplicación, 225-235

D

definición de clase
 a partir de un diagrama de diseño de clases, 299
dependencia
 notación UML, 428
desarrollo incremental, 436
desarrollo iterativo, 20, 436
diagrama
 de clases del diseño, 9, 12, 257

 adición de métodos, 260
 creación de, 255-270
 ejemplo de, 257
 información de tipos, 263
 notación de los miembros de, 268
 presentación de la navegabilidad, 264
 presentación de las relaciones de dependencia, 267
 versus modelo conceptual, 259
 y multiobjetos, 262
de colaboración, 9, 12, 169
 colecciones, 181
 componentes, 429
 condicionales mutuamente excluyentes, 180
 creación de, 172, 217-245
 creación de instancias, 177
 ejemplo de, 170
 instancias, 173
 iteración, 176
 mensaje a objeto de clase, 182
 mensajes, 174
 mensajes condicionales, 180
 multiobjeto, 181
 notación de, 167-183
 número de secuencias, 179
 parámetros de, 174
 secuenciación de mensajes, 179
 sintaxis de mensajes, 175
 valor de retorno, 175
 vínculos, 173
 y otros artefactos, 218
de despliegue, 430
de estado, 381
 acciones de transición, 388
 caso de uso, 382
 condiciones de protección, 388
 diseño de, 379-389
 ejemplo de, 384
 estados anidados, 388
 para el caso de uso, 382
 ¿para qué tipos?, 384
 sistema, 383
de estado de casos de uso, 382
de estado sistémico, 383
de implementación, 429
de interacción, 169
de paquete, 276
 creación de, 349-354
de secuencia de sistemas, 137
 creación de, 135-144
 presentación del texto de casos de uso, 143
de secuencia de sistemas y otros artefactos, 140
de secuencias, 137, 169
diagrama de clase
 diseño de, 257
diagramas de colaboración
 creación de, 405-424
diccionario de modelos, 131
diseño, 6
 de arquitectura multicapas, 271-291
 de mapeo, conversión de, en código, 295-308
 solución de Java al, 309-313
del sistema
 principios básicos, 271-291

 orientado a objetos, 6
 traducción de, a código, 295-308
 traducción de, a Java, 309-313

E

especialización, 335
especificación de las propiedades
 notación UML, 428
Esquema, 458
 búsqueda de objetos, 483
 de persistencia, 456
 diseño de, 455-486
 diseños alternos de, 484
 ideas básicas, 459
 materialización, 464
 operaciones de transacción, 479
 patrón Administración de Caché, 467
 patrón Identificador de Objeto, 461
 patrón Intermediario de Base de Datos, 462
 patrón Representación de objetos como tablas, 460
 persistencia, 456
 representación de relaciones en tablas, 475
 requerimientos, 458
estado, 379
 construcción de modelos de, 347
estereotipo
 notación UML, 428
estudio de casos de punto de venta
 introducción, 35-38
evento, 379
 asignación de nombre al, 142
 externo, 387
 interno, 387
 sistémico, 138, 206
 temporal, 387
Experto, 193
 aplicación de, 227, 230, 231
extensión, 89

F

Fabricación Pura, 396
Fachada (Facade), 418
función sistémica, 42

G

Gang of Four Patterns (patrones de la pandilla de cuatro), 405
generalización, 335, 336
 aplicación de, 335-348
 conformidad y, 338
 notación de tipos abstractos, 346
 notación UML, 336
 partición, 340
 principios básicos, 335
 pruebas de validez de subtipos, 339
 y conjuntos de tipos, 337
 y tipos, 337

glosario, 131
 elaboración de, 131-133
GoF (Pandilla de Cuatro), patrones de
 aplicación de, 405-424, 455-486
GRASP
 aplicación, 217-245, 405-424
 introducción, 185-215
 patrones residuales, 393-404

H

herencia, 348

I

inadecuación, 420
independiente del estado, 384
Indirección, 399
ingeniería inversa, 298
instancia
 notación UML, 173
Integración, 418
intensión, 89
interfaz, 102
 notación UML, 429

J

jerarquía de clase, 348
jerarquía de tipos, 335

L

Lenguaje Unificado de Construcción de Modelos, UML, 4, 10
 Véase también UML
Ley de Demeter, 401
 Véase también No hables con extraños (Don't Talk to Strangers)
límite de sistemas, 141
Lo Hago Yo Mismo (Do It Myself), 196

M

mensaje
 asíncrono, 430
 notación de UML, 427-431
 notación UML, 174
método
 abstracto, 347
 a partir del diagrama de colaboración, 302
 Fábrica (Factory Method), 473
 Plantilla (Template Method), 463
modelo, 29
 conceptual, 8, 11, 87
 adición de asociaciones, 105-118
 adición de atributos, 119-130
 conceptos adicionales, 329-333
 conceptos similares, 97

construcción de modelos de lo irreal, 98
construcción de modelos para los estados cambiantes, 347
creación de, 85-103
directrices de, 96
estrategia del diseñador de mapas, 96
introducción al, 87
obtención de conceptos, 91
organización de, en paquetes, 351
vocabulario del dominio, 88
y descomposición, 90
y diagrama de diseño de clases, 259
de análisis, 31
de casos de uso del análisis, 71
de diseño, 31
de sistemas, 30
dinámico, 30
estático, 30
sistémico, 30
multiobjetos, 181
multiplicidad, 110

N

navegabilidad, 264
No Hables con Extraños (Don't Talk to Strangers), 401
violaciones aceptables, 404
nombre de papeles (roles), 301
nota
notación UML, 427
notificación de evento, 286

O

objeto, 9
de dominio inicial, 239
de sistemas, 138, 206
operación, 101
persistente, 456

P

papel, 110
comparado con concepto, 363
paquete, 275
dependencias de, 350
notación UML, 275, 350
propiedad de, 350
referencia de, 350
paquete de arquitectura, diagrama de, 276
partición, 349
de tipos, 340
patrón, 4, 189
Agente dispositivo (Device Proxy), 400, 418
Agente remoto, 415
Agente virtual, 468
Alta Cohesión, 203
Bajo acoplamiento, 200
Comando, 214, 421
Controlador, 206
Creador, 197
de Estado, 158
Experto, 193

ÍNDICE

Fabricación Pura, 396
Fachada, 418
GRASP, 190
Indirección, 399
Ley de Demeter, 401
Lo Hago Yo Mismo (Do It Myself), 196, 394
Método Fábrica, 473
Método Plantilla, 463
No Hables con Extraños (Don't Talk to Strangers), 401
nombres, 190
Polimorfismo, 394
Publicar-Suscribir (Publish-Subscribe), 285
Puente, 469
Separación modelo-vista, 224, 281
Singleton, 413
patrones
Pandilla de Cuatro (Gang of Four), 405
de diseño
Pandilla de Cuatro "Gang of Four", 405
de GRASP
Alta Cohesión, 203
Bajo Acoplamiento, 200
Controlador (Controller), 206
Creador (Creator), 197
Experto (Expert), 193
Fabricación Pura (Pure Fabrication), 396
Indirección, 399
No Hables con Extraños (Don't Talk to Strangers), 401
Polimorfismo, 394
polimorfismo, 394
aplicación, 411
poscondición
como metáfora, 151
Principio de Hollywood, 458
proceso de desarrollo, 4, 17-27
de software, 17
fase de construcción, 25
macropasos, 20
motivación, 434
plan y fase de perfeccionamiento, 23
problemas del, 433-435
programación
con la clasificación de casos de uso, 76
orientada a objetos, 6
versiones de casos de uso, 77
Publicar-Suscribir, 285
Puente, 469

R

reingeniería de procesos de negocios, 57
relación de dependencia, 267
relaciones de usos en casos de uso, 322
requerimientos (condiciones), 41
creación de, 39-46
responsabilidad, 187
de hacer, 187
de conocer, 188
responsabilidades
patrones de, 189
patrones GRASP, 190
y diagramas de interacción, 188
y métodos, 187

ÍNDICE

asociativo, 357
notación UML, 87
partición de, 340
Véase también concepto
transición, 381

U

UML
aspectos básicos de, 15
no proceso de desarrollo, 19
notación adicional de, 427-431
Unified Modeling Language, 4, 10
Véase también UML

V

valor puro de datos, 121, 122
vínculo, 173
visibilidad, 227, 248
aspectos generales de la, 247-253
atributo de, 249
global, 252
localmente declarada, 251
parámetro, 250

Respuestas, 286
restricción
notación en UML, 427

S

Separación de modelo-vista (Model-View Separation), 224, 281
símbolo, 89
Singleton, 413
notación abreviada de UML, 415
subsistema, 349
subsistemas, identificación de, 278
subtipo
conformidad de, 338
creación de, 339
partición de, 340
pruebas de validez para, 339
supertipo
creación, 342

T

tipo, 101
abstracto, 345
notación UML, 346